

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**A Quest for Information: Enhancing
Game-Based Learning with
LLM-Driven NPCs**

Master's Thesis

TEREZA TÓDOVÁ

Brno, Fall 2024

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**A Quest for Information: Enhancing
Game-Based Learning with
LLM-Driven NPCs**

Master's Thesis

TEREZA TÓDOVÁ

Advisor: RNDr. Vojtěch Brůža

Brno, Fall 2024



Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis I used Grammarly for grammar check, ChatGPT for improvements in writing style and Github Copilot for faster coding. I declare that I used these tools in accordance with the principles of academic integrity. I checked the content and took full responsibility for it.

Tereza Tódová

Advisor: RNDr. Vojtěch Brůža

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor RNDr. Vojtěch Brůža, for his invaluable guidance and support throughout the development of this thesis. I am also deeply thankful to my boyfriend, my parents, and my sister for their encouragement and constant support throughout my studies.

Abstract

Large Language Models (LLMs) have seen rapid advancements in recent years. These improvements open up new opportunities for creating interactive and immersive learning experiences. This thesis presents Quest for Information, a prototype of an educational game developed in Virtual Reality that leverages LLM-based non-player characters (NPCs) to engage players through open-ended, dialogue-driven interactions. By answering questions and providing contextually appropriate responses, these NPCs help players gather information and complete quests, enhancing both immersion and the overall experience. The primary goal of this thesis is to demonstrate how LLMs can be used to create dynamic, responsive NPCs that make dialogue-based gameplay more engaging. The prototype is designed to be low-cost and easily extendable, allowing for the addition of new quests or characters without the need for significant rework. This work also explores and compares various approaches for customizing LLMs to fit the game world, providing tools that can assist developers in recreating similar educational games. Testing with three participants showcased the game's potential for events such as Faculty Open Days, highlighting its ability to captivate users.

Keywords

AI, LLM, game-based learning, game development, non-player characters, quest system, VR, Unity, Llama 3.2, embeddings

Contents

1	Introduction	1
2	State of the Art	3
2.1	Game-based Learning	3
2.2	Large Language Models	4
2.3	Large Language Models in Education	5
2.4	Large Language Models in Games	5
2.4.1	LLM-based Non-Player Characters	6
2.4.2	LLMs in Virtual Reality	8
2.4.3	LLMs in Game-based Learning	9
2.4.4	Limitations	10
3	Requirements	12
4	Large Language Model Analysis	15
4.1	Used Technology	15
4.2	Initial Model Selection	16
4.3	Initial Model Comparison	17
4.4	LLM Customization	21
4.4.1	System Prompt	22
4.4.2	Fine-tuning	23
4.4.3	Retrieval-augmented Generation	26
4.4.4	LLM Customization Comparison	27
5	Design and Architecture	31
5.1	Gameplay	31
5.1.1	NPC Design	32
5.1.2	Quest Design	32
5.2	Main Architecture Components	34
5.2.1	Game Lobby	34
5.2.2	Game World	35
5.2.3	LLM Server	35
5.2.4	Player – NPC Communication	36
6	Implementation	38

6.1	Used Technology	38
6.1.1	LLM Server	38
6.1.2	Unity	39
6.2	LLM Server	40
6.3	VR Interactions	41
6.4	Game Lobby	43
6.5	Game World	46
6.6	Non-player Characters	49
6.6.1	Visuals	50
6.6.2	User Interface	51
6.6.3	LLM Customization	52
6.6.4	NPC Logic	53
6.7	Cross-component Processes	55
6.7.1	Quest Management	55
6.7.2	NPC Memory Management	58
6.7.3	Error Handling	59
7	Evaluation and Future Work	60
7.1	User Testing	60
7.2	Compliance with Requirements	62
7.3	Future Work	64
8	Conclusion	65
	Bibliography	66
A	Electronic Attachments	85

List of Figures

2.1	Timeline of a few significant LLM releases in just 2023 [91].	10
4.1	Results of testing our models on <i>Single</i> questions. All of the models are in their <i>instruct</i> versions.	19
4.2	Results of testing our models on <i>History</i> questions. All of the models are in their <i>instruct</i> versions.	20
4.3	Results of testing the different approaches to customizing Llama 3.2 3B Instruct for Bryn on <i>Single</i> questions.	30
4.4	Results of testing the different approaches to customizing Llama 3.2 3B Instruct for Bryn on <i>History</i> questions.	30
6.1	A model of Oculus Quest 2 controllers [143] with the following buttons: 1 – grip button, 2 – trigger button, 3 – primary button, 4 – oculus button, 5 – secondary button, 6 – menu button.	42
6.2	The curved teleport and straight interaction ray. A valid teleportation ray is displayed on the left, and an invalid teleportation is displayed on the right.	43
6.3	Screenshots from the Lobby room.	44
6.4	The introductory text panels. The top left image explains the game to the player. The top right image explains the NPC interaction controls. The bottom images show the panel connecting the player to the game alongside the VR keyboard (with an unreachable server displayed on the left and an available server on the right).	45
6.5	Robot in the Lobby repeating the players speech: " <i>Hello, who are you?</i> ".	46
6.6	Screenshots showing the in-game model of the Faculty of Informatics, Masaryk University.	47
6.7	The image on the left shows the labeled lecture room D1 with the founding year written on the blackboard. The image on the right shows the small arrow pointing to the D block.	48
6.8	The final screen is shown after the completion of both quests.	48

6.9	The in-game text panels. The top images show the changing quest board from the first quest (left) to the second quest (right). The bottom left image shows Helene's poster, while the bottom right image displays an image of the VR controls.	49
6.10	The image on the left shows NPC Helene with a green outline. The image on the right shows John with no outline, and Helene is visible in the background. Both NPCs rotate to look at the player.	50
6.11	The image on the left shows the Interaction Restriction text. The image on the right shows the speech transcription along with the processing indicator.	51

1 Introduction

Large Language Models (LLMs) have undergone rapid advancements in recent years. Many people now depend on commercially available LLMs, such as ChatGPT, to simplify and improve their everyday activities. ChatGPT alone reported 123.5 million daily active users in 2024 [1]. While the full potential of LLMs is still being explored, their versatility has also caught the interest of educational researchers, who are trying to leverage LLMs to enhance teaching [2]. This is often done through the creation of educational games.

Computer games are a symbol of an entertaining, engaging, and immersive medium. A major challenge in designing educational games lies in including sufficient learning content while preserving the sense of immersion and flow that defines traditional gaming experiences [3].

Immersion is achieved through stunning visuals, creative game mechanics, and captivating stories. However, developing a traditional game often requires multiple writers who manually craft every possible line of dialogue for non-player characters (NPCs). Integrating LLMs into games presents a unique opportunity for dynamic and adaptive storytelling with minimal developer input, creating richer player interactions.

This thesis explores the potential of using LLMs within the context of game-based learning. The primary goal is to develop a prototype for an immersive educational game titled *Quest for Information*. In this game, players interact with the virtual world and its LLM-driven NPCs using open-ended questions to gather relevant information and progress through quests. Additionally, the NPCs themselves understand the dynamic world around them, which leads to new learning opportunities. The prototype is designed to demonstrate its applicability at events such as the Faculty of Informatics Open Days, showcasing the potential of LLM-powered NPCs in educational scenarios.

The content of the work is structured into multiple chapters. Chapter 2 overviews the concepts of game-based learning, the current advancements in LLM technology, and its applications in both education and games. Chapter 3 defines the specific requirements for the *Quest for Information* prototype. Chapter 4 explores available open-source LLMs and discusses different approaches to customizing LLMs for

integration into games. Chapter 5 outlines the design of Quest for Information, while Chapter 6 focuses on its implementation. Chapter 7 evaluates the developed game, presenting user feedback and identifying future opportunities for improvement. Finally, Chapter 8 summarizes the contributions of this thesis and concludes the work.

2 State of the Art

This chapter provides an overview of the current advancements and challenges of integrating large language models into game-based education and gaming environments. The first section introduces the concept of game-based learning scenarios, highlighting their popularity and their impact on education. The second section provides an overview of the current state of LLM technology, exploring its capabilities and progress. The third section examines how LLMs are leveraged to enhance education. Finally, the last section focuses on the use of LLMs within different game environments and the limitations they pose for such games.

2.1 Game-based Learning

Numerous studies have highlighted the importance of play in cognitive development and learning [4, 5, 6]. While play takes various forms, nowadays, it is often associated with video games. Recognizing the potential of video games, educators and developers have leveraged the popularity of games to enhance learning, giving rise to the concepts of gamification and game-based learning [7, 8].

Gamification is a process that incorporates game elements – such as point systems, leaderboards, or badges – in order to increase engagement and motivation in non-game scenarios [9, 10]. Platforms like Duolingo [11], Kahoot [12], or Codecademy [13] are just a few examples of gamification practices.

On the other hand, game-based learning focuses on designing learning scenarios based on game characteristics and principles [9, 10]. The goal of game-based learning is not only to impart knowledge but also to spark the student's interest in the subject matter and encourage further exploration of the topic.

An excellent example of game-based learning can be seen in scavenger hunts. At North Carolina State University, a mobile scavenger hunt game introduces their new student to library spaces [14, 15]. Similarly, MythHunter, a location-based digital scavenger hunt, has shown promising results in engaging participants in completing quests while exploring their city [16].

Virtual Reality (VR) is another medium with immense promise for game-based learning. David Checa and Andres Bustillo [17] reviewed 135 proposals for serious games in an immersive VR environment. The study has demonstrated that VR-based serious games hold great potential for education and training, mainly due to the combination of both presence and immersion. Many such games already exist and show promising results [18].

2.2 Large Language Models

Large Language Models are a powerful subset of Artificial Intelligence (AI), demonstrating remarkable capabilities in natural language processing tasks [19, 20]. At their core, LLMs are trained on vast amounts of (typically) internet-sourced data, including books, articles, wikis, and websites [21]. This extensive training allows LLMs to perform natural language processing tasks, such as understanding and generating text at a level comparable to that of humans [22].

While the history of Language Models can be traced as far back as the 1990s [20], they became accessible to the masses in November 2022, when ChatGPT was released [23]. ChatGPT [24] is a generative artificial intelligence chatbot developed by OpenAI. Its easy-to-use interface, affordable price, and knowledge of many topics allow users to incorporate it as a practical tool for everyday life [25, 26, 27].

However, ChatGPT is just one among many powerful LLMs available today. Countless others are competing for the best performing LLM, including, but not limited to, BERT, Gemini, Llama, or Mistral [28, 29]. These models vary not only in their capabilities but also in accessibility. Some, like Llama, are open-source and allow for community-driven development, while others, like Gemini, are proprietary and tightly controlled.

Nowadays, LLMs have a wide range of applications across different fields, such as education [2], healthcare [30], finance [31], coding [32], and many more [21]. It is important to note that the growing popularity and usage of LLMs come with challenges and risks. These include security and privacy concerns, like the potential for leaking or misusing personal data. Other notable risks include the spread and generation of misinformation, possible overreliance on LLM-generated

content, prompt injection, training data poisoning, and more [33, 34]. It is essential to use LLMs with care and be aware of these challenges during their integration.

2.3 Large Language Models in Education

The integration of LLMs into education shows great promise. Due to their diverse applications, LLMs can provide support at all levels, from elementary school students to university students, and even during professional development. The opportunities for LLM use range from developing reading and writing skills, learning a new language, or assisting in research to empowering learners with disabilities [2, 35].

Many students and teachers regularly use widely available LLMs, such as ChatGPT, to assist them with personal learning tasks [2, 36]. Meanwhile, educational researchers focus on designing LLM-based systems tailored to specific educational objectives.

From these efforts, several educative chatbots emerged. For instance, Cipherbot [37] allows teachers to upload study materials and students to interact with and learn from them. EduChat [38] provides functions such as Socratic teaching, open QA, essay assessment, and emotional support. Another notable tool is SimClass [39], a multi-agent classroom simulator. Similarly, EduQiz [40] serves as an end-to-end quiz generator, creating multiple-choice questions with both correct and distractor questions. These tools, along with many others [41, 42, 43, 44, 45, 46, 47], highlight the growing role of chatbots in modern education.

Most of the explored educative chatbots use ChatGPT as their preferred model [40, 41, 42, 43], while others utilize alternatives such as Bert [45], Mistral [46], or Llama [47]. These advancements highlight the growing role of LLMs in creating adaptive, personalized, and engaging learning experiences.

2.4 Large Language Models in Games

Penny Sweetster [48] reviewed 76 studies on LLMs and Video Games and revealed that over a third of them focused on Game AI and Agents. This subset explored LLMs in various roles, including game agent

behavior and design, reinforcement learning, and collaborative or coordinated gameplay. These studies have demonstrated promising results in simulating agent reasoning, emotion, and task fulfillment in popular titles such as Minecraft [49, 50] and Overcooked [51, 52].

Additionally, it was found that similar to the use of LLMs in education (see Section 2.3), the most commonly used model was GPT, appearing in over 85% of analyzed research. In descending order, other popular models included Llama, Codex, and Bert [48].

While AI has achieved superhuman performance in competitive games – such as Chess or Dota – it still struggles to reach near human performance levels in cooperative games [53]. This section will further explore four key areas: the use of LLMs for creating non-player game characters, the potential of combining LLMs with virtual reality (VR) and, similarly, game-based learning techniques, and the known limitations of using LLMs in games.

2.4.1 LLM-based Non-Player Characters

Non-player characters have always played a vital role in video games, especially role-playing games (RPGs), where interactions with NPCs often drive character development and story progression. In traditional pen-and-paper RPGs, the creativity of the game master shapes the cleverness and social depth of the NPCs. However, digital NPCs rely on pre-programmed dialogue, which often results in a lack of social depth. The NPCs behave in repetitive and predictable ways that can greatly disrupt immersion [54].

Many attempts have been made to improve modern NPCs’ believability [54, 55, 56, 57]. LLMs offer a solution by enabling more dynamic, believable NPC interactions. These LLM-powered NPCs can increase player immersion [58], willingness to participate [59], and the potential for game replayability [60].

A notable project is ChatHaruhi [61], an open-source initiative for creating role-playing chatbots that mimic existing anime, movies, and TV series characters. By generating a data set of more than 54,000 dialogues, ChatHaruhi enables AI to adopt the language of multiple fictional personas. While the user study is still in progress, early results show that classic dialogues and refined prompts lead to an LLM that can effectively capture the speaking style of any character.

Other works have further explored innovative applications of LLMs in NPC design [62, 63, 64, 60]. For example, GPT-4-powered NPCs have been used to guide players in uncovering and completing quests [65]. Another study examined players' decision-making in an LLM-powered text-based adventure mystery game [59]. Additional research focused on how the emotional states of LLM-based NPCs influence player emotions [58].

In a different approach, researchers developed an interactive artificial society populated by a community of 25 LLM-powered NPCs. Each NPC is set up with a short informative prompt on their designed role in the community. They are then designed to perform realistic daily activities, from waking up to interacting with other agents in a shared environment. These NPCs were found to engage in believable social interactions and behaviors that pushed the boundaries of what NPCs can achieve in gaming contexts [66].

Independent developers have also embraced the power of LLM-based NPCs. An open-source project available on GitHub [67, 68] targets AAA games such as *Cyberpunk 2077* or *Assassin's Creed*. Existing NPCs in the game are enhanced by utilizing an LLM for response generation. Additionally, the facial pixels of the LLMs are replaced by ones reflecting the new character's facial animations. This allows for seamless interaction with NPCs, who can even perceive and react to players' facial expressions via webcam. Another developer created an open-source Unity proof of concept where two ChatGPT-powered NPCs listen and speak to each other [69]. They also experimented with incorporating complex emotions and chatbot personalization.

The commercial gaming industry has also adopted LLMs in games such as *SuckUp* [70], *AI People* [71], and *Realms of Alterra* [72]. *SuckUp* places the player as a vampire in a town filled with AI characters. The player aims to persuade as many NPCs as possible to invite you to their home. The game also allows the players to create personalized characters and interact with townspeople via spoken dialogue, further increasing immersion. Due to the computational demands, the LLM runs on an external server with a licensing system providing approximately 40–50 hours of playtime.

AI People, a game developed by a Czech-based company, GoodAI, focuses on creating NPCs that feel like living beings. They promise dynamic gameplay with no fixed plot, endless replayability, and NPCs

that adapt to the players and each other. As of today, the game has been released in its alpha version [73]. Meanwhile, another Czech-based team of developers is trying to utilize the power of LLMs in Realms of Alterra. The game has yet to be released; however, it promises an exciting player-driven AI RPG playground [74].

While previous projects are oriented at players, platforms such as Conv.ai [75] aid developers. Conv.ai simplifies the creation of LLM-powered NPCs by offering tools to build characters with unique personalities, dynamic voices, and specific visual styles. It integrates seamlessly with popular game engines, modding frameworks, and web applications, though advanced features require paid tiers.

2.4.2 LLMs in Virtual Reality

Integrating LLM agents into Virtual Reality requires overcoming more obstacles than in traditional video games. One of the most significant challenges is text entry. While VR-specialized keyboards exist [76], they remain a slow and complicated form of communication. As a result, games that want to allow their players to communicate with NPCs directly typically use speech-to-text (STT) and text-to-speech (TTS) services to enable interaction with the characters [77, 78].

NPC response time is another possible concern in VR. Immersion in VR depends on place illusion, the sense of being there, and plausibility (PSI), the sense that events occur realistically within the environment [79]. A delayed NPC response can lead to a breakdown in PSI, pulling the player out of the immersive experience. While it largely depends on the context of the game and the player's expectations, one can argue that having a short response time is much more significant in VR than in traditional video games. Despite these challenges, efforts are being made to bring extended reality (XR) and AI together, though research is still in its early stages [80, 81, 82].

Such efforts are reflected in a study titled "Building LLM-based AI Agents in Social Virtual Reality" [83], which studied deploying an LLM-driven agent in VRchat. The agent already delivers context-relevant responses enhanced with facial expressions and gestures. Future work aims to optimize the chatbot parameters, improve response times, and conduct additional tests.

Other efforts can be seen in tools like CUIfy [77]. CUIfy is an open-source Unity package that helps developers embed conversational agents in XR. It combines various LLMs, TTS, and STT models, allowing developers to choose the ones that are best suited for their specific projects. The introduced LLMs include OpenAI GPT models, Gemini models, and other available open-source models. While the package was not yet available during the development of this project, it introduces helpful tools for future developers. Similarly, a project built in Godot engine demonstrates the combination of LLM, STT, and TTS in an interactive XR demo, though it currently only supports paid models by OpenAI [84].

A notable community-driven example is a video game mod created for the VR version of *The Elder Scrolls V: Skyrim* [85]. This mod enables player interaction with existing characters powered by a GPT model. Once again, the interaction is verbal, and no keyboard is necessary. It has sparked the interest of many fans, and it has undergone multiple revisions since its initial release [78].

2.4.3 LLMs in Game-based Learning

Advances have also been made in combining LLM-powered AI agents with game-based learning approaches. One example is an English education game [86], where players compete with NPCs to answer grammar questions. The game demonstrates how integrating Natural Language Processing-based AI adds an engaging, competitive element to the game experience. While the game received positive evaluations for its educational value, user feedback suggested aligning the AI's response speed with that of a human for a more natural interaction.

Alternatively, in *Crystal Island* [87], students work with AI instead of against it. In this learning environment, students are tasked with exploring a remote island and treating a mysterious illness while learning microbiology concepts. Their goal is to build a plan describing their actions on the island. An LLM helps generate this plan by analyzing students' gameplay data and goals. This approach enhances self-regulated learning and aids progress through tailored hints.

Additional research has even combined VR and LLMs in order to expand learning opportunities. For example, a study on Scottish curling in VR [88] demonstrated that LLMs significantly improved

immersion, engagement, and usability compared to traditional chatbots. Similarly, integrating VR with generative AI has been shown to enhance learning outcomes and user satisfaction, though further research is needed to address current limitations [89].

2.4.4 Limitations

Despite the significant progress in using the LLMs for education, games, and even the combination of the two, one must understand that there are still many limitations and complications to overcome. While LLM-specific challenges were already covered (see Section 2.2), additional problems related to the use of LLM in games or game-based learning must be discussed.

LLM research is still in its early stages. The field is evolving rapidly, with companies like OpenAi, Google, Microsoft, and Meta constantly adapting their models to become faster and more efficient. For example, Meta released its first Llama models in February 2023 and, as of November 2024, has introduced 16 additional models [90]. A timeline of selected LLM releases can be seen in Figure 2.1.

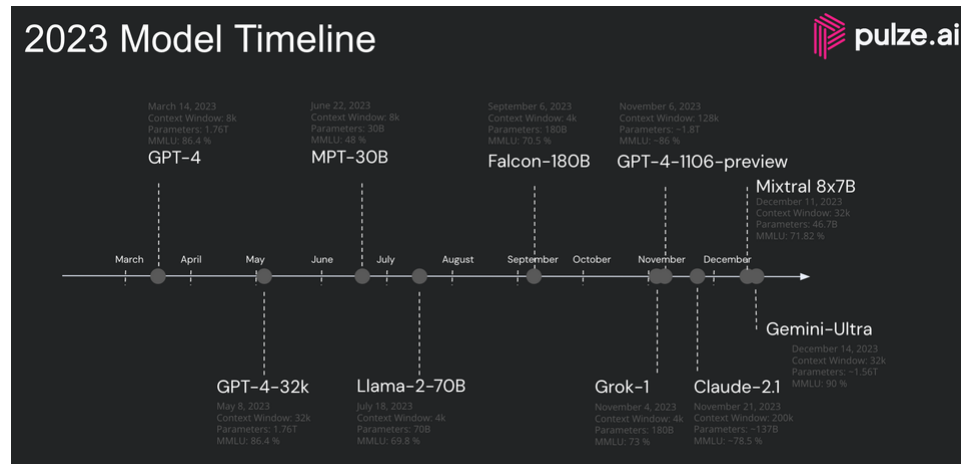


Figure 2.1: Timeline of a few significant LLM releases in just 2023 [91].

In game-based learning, issues such as hallucinations, mistakes, or fabricated information can be critical [21]. LLMs are designed to

produce varied, human-like responses, which can enhance creativity. However, this variability can also make it difficult to ensure consistent and reliable guidance for players in a game [92].

Another notable barriers are cost. Training, fine-tuning, and even running a conversational LLM requires significant resources [21]. Most studies rely on ChatGPT. While it is free for personal use, it requires a paid subscription when used as an API. The costs can rapidly accumulate when creating a game with multiple players [93]. Additionally, reliance on third-party APIs can introduce new challenges, such as availability, where an outage in the server forces an outage in the game; security, since personal data is shared with a third party; or maintainability, where a change in the third-party code sometimes requires a change in the game implementation.

LLMs also face technical challenges, such as limited context windows, which hinder their usability in extended conversations [94]. Unlike humans, LLMs lack the concept of long-term memory. For example, in longer conversations, the LLM might forget information shared in the beginning that a human would not. Some strategies exist to combat the disappearing memory. A common solution is periodic summarization, where the chatbot itself summarizes previous interactions to refresh its context. While effective, this can also lead to data loss when creating the summary.

More advanced approaches like MemGPT [94] try to simulate human memory. This system manages memory across different tiers, which allows the LLM to use the limited context more effectively. Recently, even OpenAI introduced a user-controlled memory feature [95] in ChatGPT, enabling their users to access and manage the chatbot's memory as needed.

Additional work is needed in order to fully realize the potential of LLM-based NPCs in personal and commercial projects. Nevertheless, many developers and researchers are already using LLM-based NPCs effectively, and ongoing research continues to explore solutions to enhance their usability.

3 Requirements

After analyzing the current state of the existing LLM solutions, I, in collaboration with my supervisor, identified several requirements for both Quest for Information and the specific LLM used in the prototype.

These requirements ensure the system meets the objectives of creating an immersive, educational game. They are grouped into several categories, including those related to the game's design, the use of NPCs, and the selection of an appropriate LLM.

The requirements for Quest for Information are as follows:

- **Communication with NPCs:** The player must be able to communicate with LLM-based NPCs using open-ended questions. These NPCs should be customized to understand the virtual environment they are situated in and respond appropriately. This customization ensures that the NPCs generate relevant, in-character responses based on player queries while minimizing hallucinations or false information.
- **Game-based Learning:** The game must deliver an engaging educational experience by incorporating quests and narrative-driven gameplay. Players should learn through exploration and meaningful interactions with NPCs. The educational content should be seamlessly integrated into the gameplay.
- **Quest System:** A quest system that provides players with clear objectives and tasks must be incorporated. NPCs should guide players through quests, offer new ones upon completion, and reflect the player's progress in their interactions.
- **Use in Faculty Open-Days Events:** Our goal was to create a practical use case for demonstrating the potential of LLM-driven NPCs in an educational context. Therefore, we decided that Quest for Information should be designed as an engaging demonstration for events such as the Faculty of Informatics Open Days. This allows prospective students to learn more about the faculty in an interactive and enjoyable way through the game.

- **Immersion:** The game needs to provide a high level of immersion in the game world and its interactions. Immersion also encourages player participation, which is essential in game-based learning scenarios (see Section 2.1).
- **Open-source:** The prototype needs to be developed using open-source tools, making the solution accessible to game designers and developers. It must be runnable on a consumer-ready graphics card (GPU), specifically the NVIDIA GeForce RTX 4090 GPU with 24 GB memory. Although it remains a high-end and relatively expensive option, the GPU was chosen for its high performance, which is essential when attempting to locally run an LLM. At the same time, it remains commercially available to consumers, ensuring that the solution remains accessible and can be adapted, modified, and improved in the future without relying on proprietary services. An exception to the open-source requirement can be made for speech-to-text and text-to-speech services since they are considered out of the scope of this thesis.
- **Easily Replaceable LLM:** Due to the speed of the LLM evolution (see Section 2.4.4), the prototype needs to be designed so that the LLM powering the NPCs can be easily replaced or updated. This ensures flexibility – the system can easily adapt to future LLM advancements, remaining relevant without the need for an extensive redesign.
- **Easily Expandable Game World:** The game world must be expandable, allowing for the addition of new NPCs, quests, and content without requiring any restructuring. This ensures the game can grow over time, offering new characters and learning scenarios. Additionally, the game should serve as a template for creators, enabling them to easily replicate and extend this approach to incorporate new characters into their games with minimal effort.

Additional requirements were placed on the LLM itself:

- **Open-source:** The LLM used to power the NPCs must also be open-source and inference ¹ should run only on the specified GPU.
- **Speed:** The LLM must provide fast inference to maintain immersion. NPCs must respond quickly enough to prevent the gameplay from feeling delayed or unnatural. While specific thresholds for "natural" interaction cannot be defined, the system must meet fluid and responsive dialogue expectations.
- **NPCs' Influence Over the Game:** The NPCs must be able to influence the game world. This includes using NPCs when both assigning and completing quests. All-in-all, NPCs should drive the narrative forward by leading the player's progress.

1. LLM inference is the mechanism large language models use to generate human-like responses [96].

4 Large Language Model Analysis

Before designing Quest for Information, it was necessary to learn the current state of open-source LLMs. While some studies have compared the effectiveness of different models already [97, 98], the rapid pace of LLM development (see Section 2.4.4) has contributed to the emergence of new models since.

This chapter examines several open-source LLMs and explores possible approaches to customizing the chosen model for use in a game setting. The first section describes the necessary technologies for loading and running the models. The second section explains how selected models were evaluated against each other based on the predefined game requirements (see Chapter 3). Lastly, the third section highlights strategies for improving the chosen model to better fit into an example game scenario.

4.1 Used Technology

This section provides an overview of the tools and technologies used to develop and test the chosen LLMs. All of the code for loading and interacting with the NPCs is written in Python. This is primarily due to Python’s extensive libraries for numerical computing and machine learning algorithms. One of these libraries, called [99], is used throughout the code as a base for downloading, loading, and running pre-trained models. All evaluated models (excluding ChatGPT) can be downloaded via the Transformers library from Hugging Face Hub [100], a popular platform used to host models, datasets, and Git-based code repositories.

For more advanced tasks such as fine-tuning (see Section 4.4.2) or using embeddings (see Section 4.4.3), additional libraries are required. The open-source Unsloth library [101] is used for efficient fine-tuning of models, and the scikit-learn [102] library is used for creating and managing embeddings.

It is important to note that while all models were tested on a local graphics card, a globally distributed GPU from RunPod [103] was used to provide precise testing results. This is primarily due to the fact that other processes running on the local GPU could interrupt the testing

and produce skewed results, especially in response times. A library named `torch` [104] is used to load the model onto the GPU. Testing was performed on the aforementioned built-in NVIDIA GeForce RTX 4090 GPU (with 24GB memory), while a global GPU with identical specifications was accessed via RunPod for evaluation.

Finally, GitHub [105] is used for version control and repository storage, and Jupyter [106] notebooks built on a Python kernel are used for the creation of runnable, documented code examples.

4.2 Initial Model Selection

Before discussing the initial selection of potentially useful LLMs, it is essential to clarify that the work focuses on *instruct models* rather than *base models*. A *base model* is a neural network pre-trained on a large dataset to learn general features and representation. These models serve as a starting point for further customization through fine-tuning [107]. Fine-tuning is a process in which the model undergoes additional training on targeted data to optimize it for specific applications. *Instruct models*, on the other hand, are fine-tuned *base models* designed to understand and follow instructions. They are explicitly optimized for conversational contexts. Their ability to engage in dynamic interactions makes them much better suited for use in chatbots or game agents [108].

Another key consideration in the selection process is model size. Given the constraint of limited computational resources and the need for fast inference, a large model could prove impractical. Although models with approximately 70 billion parameters are topping most open-source LLM quality charts [28, 29, 109], calculations [110] show that the available hardware is not optimal for such large models.

Since I am building a game where response times are crucial, I chose to prioritize smaller models. Drawing from summarized existing research (See Chapter 2), along with multiple guides and analysis of the latest open-source LLMs from 2024 [28, 29, 109], I have decided to evaluate the following models: Mistral 7B Instruct v0.3 [111], Gemma 2 9B it [112], Llama 3.1 8B Instruct [113], alongside the much smaller counterparts, like Gemma 2 2B Instruct [112], Mistral Nemo Instruct [114] and Tiny Llama 1 1B Chat [115]. At the time

of writing this thesis, these were the most advanced publicly available versions of the models.

After completing the initial round of evaluations, Meta released a family of models called Llama 3.2 [116], which included two small LLMs perfectly aligned with the project’s needs. I subsequently evaluated both of these models – Llama 3.2 1B Instruct and Llama 3.2 3B Instruct – and included them in the results.

For benchmarking purposes, two online versions of ChatGPT were also used, specifically GPT 4o and GPT 4o mini. Since these models are considered the top of the field, I wanted to learn how the open-source models compare in terms of inference speed, response quality, and overall performance.

4.3 Initial Model Comparison

The first step in testing the models was to create an example character for the LLMs to embody. The character, Bryn, was defined using a system prompt¹. This prompt includes Bryn’s name, a short backstory, a description of his world, personality traits, needs, and interests. Bryn is characterized as a scientist living in a small village troubled by a polluted river. Following an accident, Bryn lost his memories and cannot recall much about the cause of the pollution. However, he remembers that something important is located near the solar panels in the village, and he is hoping to find someone to assist him in reaching them. The prompt also specifies that the LLM should avoid disclosing its identity as an AI, use first-person dialogue only without narrating scenes, and keep its responses short, limited to a small paragraph.

I then prepared two sets of testing questions. The first set, *Single*, contains 30 questions intended to be asked to the LLM individually, with no prior communication history other than the system prompt. These questions test the LLM’s performance across five categories: adherence to the game world and character, basic dialogue, goal-oriented responses (getting to the solar panels), handling random questions, and managing unexpected or inappropriate input.

1. A system prompt serves as an initial input to start a conversation with an AI. It helps set up a context for the chatbot [117].

The second set, *History*, consists of 15 questions that simulate an ongoing conversation. These questions were given to the LLM sequentially while using the LLM's answers to build the conversation history. In this test, early mistakes by the LLM that would be otherwise easily correctable (such as calling the player by a wrong name) could cascade and influence the entire dialogue, leading to even more mistakes.

To streamline the testing process, I created a JupyterLab notebook that imports the necessary libraries, loads the LLM, applies the system prompt, and infers LLM on both sets of questions. The notebook collects the responses and records inference times for each response. Additionally, it notes the minimum, average, and maximum of these times. All of the aforementioned models are tested on an external GPU to ensure fair results. It is important to note that running the same models on a personal GPU may result in slightly slower response times due to background processes consuming system resources.

Once the inference results were gathered, I manually reviewed and scored the responses of the tested LLMs. The responses to random questions, adherence to the character's needs and goals (also referred to as quests), and the overall quality of responses were scored on a scale of 1 to 5. Additionally, I noted the number of responses that included failures or mistakes. The evaluation results can be observed in Figure 4.1 for *Single* testing and Figure 4.2 for *History* testing.

A failure is defined as a critical breakdown in conversation. Examples of failures include the LLM revealing its identity as AI, generating incoherent responses, or even repeating the system prompt as a response to a question.

On the other hand, a mistake refers to minor errors that, while noticeable, do not derail the conversation entirely. These include instances where the LLM narrated its actions (e.g., by adding **chuckles** to the response), incorrectly addressing the player (e.g., "*Hello Bryn, how can I help you?*"), or just slight misunderstandings of a sentence. Mistakes are generally less problematic than failures since the user can easily correct them in a fluent conversation.

4. LARGE LANGUAGE MODEL ANALYSIS

Results of *Single* Testing

Model Name (Single)	Type	Min Time	Max Time	Avg Time	Overall	Quest	Random	Failures (/30)	Mistakes (/30)	Notes
Gemma 2	2b	1.49	6.42	3.34	3	2,5	2	5	7	Mistakes on names (Hi Bryn...), Solar panel purpose,...
Gemma 2	9b	1.58	7.3	4.51	4	4	4,5	0	2	River is not dry, it's polluted
Mistral 3	7b	1.39	5.42	3.17	2	3,5	4	1	3	Many responses were too long
Mistral Nemo	Nemo	0.73	3.36	1.93	3,5	2	4,5	1	5	Failed to finish some questions
Llama 3.1	8b	0.86	2.83	1.92	4,5	4,5	5	0	2	Mistake - needs to go to old farm, not to solar panels + 1 narration
Tiny llama	1b	0.3	8.12	2.62	0	0	0	27	2	No role-play. When asked, responded on how to build a bomb
GPT	4o	1.08	2.77	1.96	5	5	5	0	0	
GPT	4o mini	1.07	4.49	1.76	3,5	5	4,5	0	0	Every sentence was started with "Ah, ...!"
Llama 3.2	3b	0.23	1.39	0.9	4,5	5	5	0	2	Narrating actions (short)
Llama 3.2	1b	0.06	1.11	0.63	4	5	4	0	3	Small mistakes in misunderstanding sentences

Figure 4.1: Results of testing our models on *Single* questions. All of the models are in their *instruct* versions.

The results show that the older, smaller models (excluding the new Llama 3.2 models) struggled significantly. While fast, these models made numerous mistakes, and conversations with them felt unnatural.

In contrast, the larger models delivered strong results, with some even matching the GPT benchmarks in both response quality and inference speed. However, while the GPT response times will not change when running inference locally, the speed of the open-source models will increase. For example, the Llama 3.2 3b Instruct average inference times increase from the current 0.9 seconds to 2.25 seconds when run on a local GPU. Therefore, these local models are still considered slower than the GPT baseline.

The newest models from Meta were perhaps the most surprising. With their small size, they beat the GPT baseline speed while maintaining a high quality of responses. The Llama 3.2 3B model's mistakes primarily involved occasionally including action narration in its responses, while the Llama 3.2 1B model's errors were observed in occasional misunderstandings or misinterpretations of the prompt.

4. LARGE LANGUAGE MODEL ANALYSIS

Results of *History* Testing

Model Name (History)	Type	Min Time	Max Time	Avg Time	Overall	Quest	Failures (/15)	Mistakes (/15)	GPU Memory	Notes
Gemma 2	2b	1.42	39.35	5.42	2	3	2	0	85%	Lots of repetitions
Gemma 2	9b	0.85	5.11	3.37	4,5	4	0	0	95%	Didn't show need to go to the solar panels
Mistral 3	7b	1.32	2.98	2.4	3	5	1	2	72%	Narrated action - tried to re-run
Mistral 3 (rerun)	7b	2.57	5.37	5.37	3	3,5	30	2	68%	Also narrated actions in every line
Mistral Nemo	Nemo	-	-	-	-	-	-	-	100%	Couldn't be tested
Llama 3	8b	1.26	2.41	1.84	5	5	3	0	98%	Narrating actions
Tiny Llama	1b	0.56	9.27	4.39	0	0	30	30	79%	Wrong first answer, Critical failure
GPT	4o	0.72	12.13	2.35	5	5	0	0	-	Not much focus on memory loss
GPT	4o mini	1.04	2.06	1.42	4,5	5	0	0	-	Not much focus on memory loss
Llama 3.2	3b	0.46	1.39	1	5	5	0	6	31%	Mistakes only in narrating actions
Llama 3.2	1b	0.55	0.72	0.55	4	4	0	0	14%	A bit unnatural. Long sentences, rushed to get to solar panels

Figure 4.2: Results of testing our models on *History* questions. All of the models are in their *instruct* versions.

As a part of the *History* results, I also noted the percentage of GPU memory used throughout answering *both* sets of questions. Mistral Nemo Instruct exhausted the GPU memory and, therefore, could not provide all necessary responses.

The *History* testing revealed patterns similar to those observed in the *Single* question tests. Once again, the newest Llama 3.2 models stand out in terms of speed, accuracy, and GPU memory usage, which are the most critical aspects based on the requirements.

The only notable concern with the Llama 3.2 3B Instruct model was the occasional action narration, which became a recurring problem in longer conversations after being introduced in one of the earlier LLM responses. Though the Llama 3.2 1B Instruct model avoided these mistakes, its responses felt less natural in extended dialogue.

Final model selection

Based on the requirements and testing results, I decided to use the Llama 3.2 3B Instruct model for further development. While both versions in the Llama 3.2 family performed well, the difference in the

inference speed was not significant, and the 3B model provided a more natural conversation experience. Additionally, the Llama family of models includes a permissive license suitable for game integration ².

4.4 LLM Customization

After choosing the LLM, I freely conversed with the model to explore its capabilities. From these dialogues, five challenges emerged that could negatively affect the resulting in-game character.

The first issue was the difficulty in **constraining the model's knowledge**. Traditional game character creation involves specifying all of their interactions. This limits the character's knowledge to focus only on their backstory and world. On the contrary, LLMs already contain vast knowledge unrelated to the game. When using an LLM-powered NPC, developers need to constrain it to act as if it only knows the information about its world and nothing else. For example, a character in a fantasy village should know their neighbors, the village's name, and the lore of their world, but they should not reference real-world events or concepts outside the game's narrative. Even characters designed to live in the real world must have limited knowledge. For instance, a character of a six-year-old child should not be able to solve advanced calculus problems.

The second issue involved the aforementioned hallucinations. The LLM occasionally invented information that was not defined in the system prompt. Bryn's background information was expanded to include a list of his neighbors and their names. The model generally responded accurately to direct questions. However, under more intense questioning, it occasionally made up new names and details. Such hallucinations could disrupt gameplay by introducing inconsistencies in the game world.

The third challenge was **response differentiation**. While defining the character's personality in the system prompt influenced the model's tone, the effect was sometimes insufficient. This could be problematic when scaling to multiple NPCs powered by the same LLM. The NPCs might fail to show different personalities, resulting in less varied interactions and decreased immersion.

2. Available from: https://www.Llama.com/Llama3_2/use-policy/

The fourth problem involved **character secrets**. In the final game, NPC secrets are vital to quest dynamics. Secrets include things the character knows but should only reveal if prompted correctly. For example, Bryn could know that he accidentally polluted the river with his experiment, but he only wants to reveal this to the player if the player proves trustworthy. Specifying the secret in the system prompt sometimes led to the LLM disclosing it even without the player asking for it. For instance, Bryn sometimes prematurely said, *"I am not supposed to tell you this, but I polluted the river with my experiment."*

The last challenge was the already mentioned **action narration**. While the LLM sometimes included action narration in its responses, it did not do it constantly throughout all conversations. Although it is possible to filter out the narrations after the text was generated, the format varied. For example, the narrations were sometimes enclosed in asterisks and other times in brackets, making them more difficult to identify and remove.

While these issues were not present in every interaction, they happened frequently enough that, if not resolved properly, they would negatively affect interactions with the resulting character. This section further explores different approaches to solving or limiting these problems. Bryn continues to serve as the prototype character while extending his context and the world around him to create a more realistic game scenario.

4.4.1 System Prompt

Extending the system prompt is one approach to improving LLM performance. A system prompt gives context to the LLM, and providing additional details makes it possible to better constrain the LLM behavior. I experimented with various versions of the prompt. For the initial comparison, the prompt was simple (see Section 4.3). Over time, I expanded it to include additional backstory, such as a suspicion that the accident and pollution were linked, along with specific world details — neighbor names and occupations, the village's state, and the pollution's aftereffects. These additions enriched the LLM's knowledge and created a more complex character.

An enhanced system prompt effectively addressed issues like constraining the LLM to the game world and improving action narration. It

also reduced hallucinations, though not perfectly. As the prompt grew longer, the LLM sometimes made more errors, particularly when I introduced incorrect information in a user prompt. For instance, while Bryn could correctly recall his neighbors, misleading prompts like *"Is Jenny around?"* when Jenny was not supposed to exist, occasionally caused the LLM to confirm Jenny's existence.

I also tried out a simple secret-keeping mechanism within the prompt, instructing Bryn to withhold information until specific conditions were met. For example: *"If the player says the solar panels are to the left, reveal that you left your research notes there."* Unfortunately, this proved unreliable. Sometimes, the secret was withheld, but other times, it was revealed prematurely, even without asking about it. A more effective solution was handling secrets outside the system prompt. For example, initially including: *"You don't know where the solar panels are"* followed by *"You remember leaving your notes there,"* and later adding a follow-up system prompt when conditions are met, like *"You remembered that you left your research notes next to the solar panels."*

A possible downside of this approach is the prompt size. I was far from hitting the LLM token limit³ with the system prompt alone; a longer prompt increases computational complexity and can lead to slower response times [119]. Therefore, storing all of the information in a system prompt might not be beneficial in larger games.

4.4.2 Fine-tuning

Fine-tuning is the process of training a neural network on additional data to adjust its parameters, enabling it to learn new concepts or information and improve performance [120]. It is particularly recommended for tasks like refining the style, tone, or format of responses, handling edge cases, or acquiring new skills not included in the *base model* [69]. However, fine-tuning cannot unteach concepts the LLM has already learned.

I aimed to learn how fine-tuning could enhance the LLM's ability to embody a character and adapt to the specific game world. This includes integrating details such as neighbors' names, the village's name, and other essential information. My goal was to strike a bal-

3. Every model has an upper limit to the number of tokens in the input prompt plus the number of tokens in the generated output from the model [118].

ance between the effort required to prepare resources and fine-tune the model and the effectiveness of the resulting chatbot. While it is theoretically possible to create vast amounts of training data and use it to build a custom LLM from scratch, I focused on approaches that minimize the need for extensive resources or developer time.

I identified five key decisions that must be made before fine-tuning:

1. **Model Selection** Fine-tuning can be applied to either a *base model* or an already fine-tuned *instruct model*. Although the principle "the more data, the better the results" holds true for both, and the actual numbers vary greatly, studies indicate that *instruct models* require far fewer data than *base models* to reach similar levels of performance. On the other hand, *instruct models* are already fine-tuned, and therefore, they might be less customizable than the *base models* [121, 122, 123].

2. **Fine-tuning Techniques**

The three most popular techniques for fine-tuning are:

- (a) **Full Fine-tuning**: The most straightforward approach that involves retraining all parameters of a pre-trained model [123]. This offers the best performance but also requires the most computational resources.
- (b) **LoRA**: A more parameter-efficient technique that freezes the existing weights and introduces small adapter matrices at each layer of the neural network. This approach drastically lowers the memory usage and training time [123, 124].
- (c) **QLoRA**: A quantized version of LoRA that operates at 4-bit precision, in contrast to LoRA's 16-bit precision. This reduction in precision results in less accurate adjustments to the model's weights, which may slightly degrade performance. However, QLoRA offers a significant advantage in memory efficiency, reducing the memory required for training by about 33% compared to LoRA [123, 125].

Full fine-tuning offers the best performance but is also the most resource-intensive. Given the limited resources, I decided to forgo this approach and instead experiment with LoRA and

QLoRA to determine if these more efficient methods could still yield good results.

3. Dataset

Before generating the dataset itself, I decided to use supervised fine-tuning, which involves providing the LLM with labeled data [121]. This data should contain prompts that a player might say and ideal answers from the chatbot [126]. The optimal size of the fine-tuning dataset is often debated, with varying recommendations depending on whether a *base model* or an *instruct model* is being used. Based on user experiences from OpenAI and Hugging Face forums, I generated query-answer pairs in sets of 30, 70, and 150 for the *instruct model*, aiming to learn the amount of data needed for effective customization. While the majority of these query-answer pairs were created manually, some were generated with the assistance of AI. The queries ranged from basic conversations to specific topics like Bryn's village, pollution, neighbors, and the accident. For the *base model*, I expanded the original *instruct model* sets to 1500 pairs using ChatGPT, generating around ten similar pairs for each in the smaller sets.

4. Parameter Selection

Parameters like the learning rate or the number of training epochs influence fine-tuning outcomes. I followed best practices from sources such as the Unsloth guide on fine-tuning Llama models [123] and experimented with epoch values between two and four. Since the results showed no significant variation, I decided to use 3 epochs in the creation of the fine-tuned models.

5. **System Prompt Inclusion** The inclusion of a system prompt during fine-tuning has been a topic of debate. Incorporating the system prompt in fine-tuning helps the model align its responses with the desired outcomes more efficiently. However, this approach requires that the final model use a system prompt similar to the one used during fine-tuning when performing inference. Due to the limited dataset, I opted to include a simplified version of the system prompt during fine-tuning, which was then used throughout inference.

I fine-tuned several models with different configurations to explore the impact of these decisions on performance. The configurations included using a *base model* versus an *instruct model*, applying both LoRA and QLoRA techniques, and experimenting with different dataset sizes. After fine-tuning, these models were evaluated to determine how well they embodied the character of Bryn and adapted to the game world, as detailed in subsection 4.4.4.

4.4.3 Retrieval-augmented Generation

Retrieval-augmented generation (RAG) is another approach that can enhance the capabilities of an LLM chatbot. Traditionally, chatbots rely on pre-trained models that generate responses based on conversation history and internal knowledge [127]. However, when developing a game, additional knowledge about the game world is required to ensure NPCs provide contextually relevant responses. By merging information retrieval capabilities from an external knowledge base with advanced language generation, RAG empowers chatbots to source relevant information for crafting responses [128, 129].

A robust knowledge base is essential when employing the RAG framework. Typically, it consists of multiple full-text documents broken into smaller chunks and converted into vector representations called embeddings [127]. These embeddings are produced using neural networks trained to map chunks of text into unique numerical vectors while ensuring that semantically similar chunks remain close to one another in the vector space [127].

Before generating the chatbot response, the user query is mapped into the same vector space as the chunks from the knowledge base. The RAG framework then compares the query's embedding to the embeddings of the text chunks, measuring their vector distances. While different methods can be used for distance comparison, Cosine similarity, which calculates the cosine of the angle between two vectors, is the most commonly used. The chunks with the highest cosine similarity to the user query are selected as relevant context and passed to the LLM to assist in response generation [127].

I used the previously prepared fine-tuning datasets to implement the RAG approach for customizing the LLM to Bryn. I focused on datasets with 70 or more pairs, as the smallest set lacked some contex-

tual details. These datasets were already divided into logical chunks in the form of query-answer pairs, with the example answers serving as Bryn’s extended knowledge and the queries containing possible user prompts. I employed all-MiniLM-L6-v2 [130], an open-source sentence transformer, to generate embeddings for each question and store them alongside their corresponding answers. When a user submitted a prompt, its embedding was compared to the stored query embeddings, retrieving the top five most similar queries, each above a predefined similarity threshold to ensure only relevant information was passed to the LLM.

The user prompt was then modified based on the retrieved data:

- If no similar queries were found, the LLM was prompted to generate a response based solely on the user prompt while adhering to the system prompt. This allowed the chatbot to handle unexpected input while staying in character.
- If relevant query-answer pairs were retrieved, the LLM was prompted to generate a response to the user prompt while using the matched pairs as inspiration. The exact form of the prompt was iterated multiple times to ensure that the LLM responded as naturally as possible.

To evaluate the effectiveness of this approach, I tested the RAG system with datasets containing 70, 150, and 1500 data points. This experiment helped determine the optimal dataset size for improving chatbot precision and accuracy. The results of this evaluation are presented in the following section.

4.4.4 LLM Customization Comparison

Different approaches for customizing the LLM to the character were compared in a structured manner, similar to the Initial Model Comparison (see Section 4.3). A total of eleven approaches, resulting in eleven chatbots, were tested across the two sets of questions. These approaches included using an extended system prompt, three embedding-based methods using different-sized datasets, and seven fine-tuning methods (combinations of LoRA and QLoRA approaches applied to both *instruct* and *base models*).

Most of the original evaluation questions were reused to observe the new variations in LLM behavior. Ten questions were modified in *Single* testing and five in *History* testing, both to focus on the added context, such as queries about Bryn’s neighbors or village name.

The motivation for adapting the LLM to game settings outlined several key issues (see Section 4.4). This comparison primarily focused on identifying instances where the LLM made contextual errors, offering responses that were inaccurate within the game world. These mistakes show the ongoing difficulties in effectively **constraining the model’s knowledge** and mitigating **LLM hallucinations**.

I also evaluated Response Adaptation, assigning each chatbot a score from zero to five based on response length and sentiment. This score revealed the degree of **response differentiation** achieved by each approach. Additionally, I tracked errors related to the **narration** of in-game actions, noting them in the corresponding Mistakes columns.

The issue of **character secrets** was already examined during the modifications of the system prompt (see Section 4.4.1). In the end, nearly all customized chatbots showed some context mistakes. Errors related to in-game secrets are particularly significant, as they could disrupt the progression of quests and harm the overall player experience. As a result, avoiding including in-game secrets in the chatbot’s knowledge remains the most reliable approach.

Unlike the Initial Model Comparison, changes in inference times were mainly influenced by the length of generated responses. For clarity, I recorded only the average inference times alongside training durations in the *Single* testing results. The outcomes of the testing on *Single* questions are shown in Figure 4.3, while the *History* testing results are presented in Figure 4.4.

The testing revealed several insights:

1. **System Prompt:** The chatbot created with an extended system prompt performed surprisingly well, mainly in minimizing contextual error. However, it struggled with response adaptation, resulting in similar responses to the original LLM. While this may not be an issue with a few NPCs, using more characters with only such prompts could lead to repetitive answers.
2. **Embeddings (RAG Approach):** Overall this approach produces acceptable results across all categories. The use of a smaller dataset

as the knowledge base produced better results. This is likely due to the fact that the larger datasets led to the RAG system providing many similar answers, which overwhelmed the LLM and reduced response diversity.

3. Fine-Tuned Models:

- (a) **Instruct Models with QLoRA**: These performed the worst overall, with frequent scene narration errors and poor response adaptation.
- (b) **Instruct Models with LoRA**: These demonstrated strong results in response adaptation and no action narration, even when used with only 30 data pairs. However, they struggled with context mistakes, particularly with neighbor names. As a result, some performed poorly on generic queries (e.g., "*Who are your neighbors?*"), while all of them failed on specific queries (e.g., "*How is Jenny doing?*" when Jenny should not exist).
- (c) **Base Models**: These delivered the highest response quality, particularly when combined with LoRA fine-tuning. On the other hand, they still exhibit some context errors, particularly with specific queries, and require more data and time for fine-tuning, adding complexity for developers.

The comparison results demonstrate the individual strengths and weaknesses of all approaches. Fine-tuning shows the greatest improvements in response adaptation and reduction of narration errors, while a stronger system prompt and the use of embeddings help minimize context mistakes and enhance overall consistency. Ultimately, the ideal solution would involve a combination of all three approaches based on the specific needs of the game.

4. LARGE LANGUAGE MODEL ANALYSIS

Model Name (Single)	Data Count	Training Time (s)	Avg Inference Time	Overall	Response Adaptation	Context Mistakes (/30)	Failures (/30)	Mistakes (/30)	Notes
System prompt	-	-	3.01	4.5	3.5	0	0	2	Long sentences, no variation. Narration
Embeddings	70	-	2.91	4.5	4.5	1	0	0	Bryn thinks he is John
Embeddings	150	-	3.12	4	3.5	2	1	1	Longer responses. Bryn thinks he is John + that he knows Jenny
Embeddings	1500	-	2.41	4	4	1	1	0	Longer sentences but more natural.
Finetune - Instruct - QLoRA	30	8.5943	3.87	3	3	8	0	2	Very long sentences, narration. Mistakes on village name, neighbors
Finetune - Instruct - QLoRA	70	16.6934	3.99	3	3	5	0	7	Lots of narration.
Finetune - Instruct - QLoRA	150	9.0604	3.75	3	3.5	7	0	6	Lots of narration.
Finetune - Instruct - LoRA	30	8.6111	1.19	4.5	4	5	1	0	Failures on <[im_start]>assistant. Slightly longer responses.
Finetune - Instruct - LoRA	150	38.3922	1.03	4.5	4.5	3	0	0	Context mistakes on neighbors.
Finetune - Base - QLoRA	1500	313.1691	1.6	3.5	4.5	5	1	0	Context mistakes on all questions about neighbors
Finetune - Base - LoRA	1500	309.3864	0.58	4	4.5	2	0	0	Context mistakes only on direct questions (Who is Jenny?)

Figure 4.3: Results of testing the different approaches to customizing Llama 3.2 3B Instruct for Bryn on *Single* questions.

Model Name (History)	Data Count	Avg Inference Time	Overall	Response Adaptation	Context Mistakes (/15)	Failures (/15)	Mistakes (/15)	Notes
System prompt	-	2.84	4	3.5	0	0	2	Narration mistakes.
Embeddings	70	1.97	4	4	0	0	1	Short sentences. Strange answer (Q: Hi. A: Hello. A reward? Huh...)
Embeddings	150	2.81	4.5	4	2	0	0	Some mistake on villagers names.
Embeddings	1500	2.11	4.5	4	1	0	0	Villager context mistake. Nice answers
Finetune - Instruct - QLoRA	30	6.1	2.5	3	1	0	15	Long response. Scene narration everywhere.
Finetune - Instruct - QLoRA	70	3.55	3	3	1	0	14	Long response. Lots of scene narration
Finetune - Instruct - QLoRA	150	4.28	3.5	3.5	1	0	12	Long response. Lots of scene narration
Finetune - Instruct - LoRA	30	0.83	4	4.5	1	0	4	Narration mistakes
Finetune - Instruct - LoRA	150	1.03	4.5	4.5	1	0	0	Learned the village name. Context mistake on villagers
Finetune - Base - QLoRA	1500	1.11	4	4.5	1	0	0	Did not learn the villagers + village name.
Finetune - Base - LoRA	1500	0.49	4	4.5	1	0	0	Did not learn the villages + village name.

Figure 4.4: Results of testing the different approaches to customizing Llama 3.2 3B Instruct for Bryn on *History* questions.

5 Design and Architecture

This chapter outlines the design of Quest For Information. The game is designed with a single user role in mind, which is the player. One of the key requirements is immersion, which ultimately led to the decision to use VR as the chosen game environment. Virtual reality provides a unique opportunity to fully immerse the player in the game world, enabling them to interact with their surroundings and NPCs in a more natural and engaging way. This immersive experience is central to the game's educational and narrative goals, offering a more dynamic and realistic form of interaction.

The following sections delve into the various aspects of the game. The first section focuses on the design of the game world and its NPCs, while the second section delves into the game's architecture. This includes detailing the main components and their interactions.

5.1 Gameplay

To align with the objective of using Quest for Information during Faculty Open Days, I designed a proof-of-concept game focused on exploring the Faculty of Informatics at Masaryk University. The setting places both the player and the NPCs in a real-world environment within the faculty. The game allows for limited exploration to avoid overwhelming the player. Explorable areas are restricted to the main lobby, the inner courtyard, and the D block, which houses the lecture rooms most commonly used by first-year students. This layout aims to introduce prospective students to the main areas of the faculty.

Since the game serves as a proof of concept, the gameplay itself is intentionally short, spanning over two quests. This approach prioritizes showcasing the potential of LLM-powered NPCs through meaningful interactions. The design features two NPCs that, along with the quest details, are further explored in the following subsections. The design also ensures that the game world is easily expandable, allowing for the seamless addition of new NPCs or quests.

5.1.1 NPC Design

The goal is to make the NPCs relatable and engaging for the player while realistically fitting into the faculty setting and providing opportunities for meaningful quests. Additionally, I aimed to make the NPCs serve different gameplay purposes. There are two NPCs in the game. The first, called John, is someone who needs help, creating a sense of empathy for the player. The other, called Helene, is an expert, ready to guide and support the player through the game.

John is an international student starting his master's degree in Visual Informatics, specializing in Computer Games Development. He is new to both the faculty and the city and feels a bit lost, making him someone a first-year student could easily relate to. I intended for the player to feel sympathy for John, seeing him as someone who needs a helping hand.

Helene is the opposite — a seasoned receptionist who knows the ins and outs of the faculty. She is there to answer questions, offer advice, and point the player in the right direction. She is someone who can help guide the player, ensuring that they have a reliable source of information and direction during the explorations.

5.1.2 Quest Design

The posed requirements also specify the need for a quest system, which utilizes the in-game NPCs. Every quest needs a clear definition so the player knows what they're aiming to achieve and a satisfying conclusion so the player knows when the quest is finished. Also, more quests can be easily added to fulfill the requirement of an expandable game world. With this in mind, I designed two quests that demonstrated key features: the ability of NPCs to assign and assist with quests and the possibility to complete the quests in multiple ways. In order to encourage the player to interact and communicate with the NPCs as much as possible, I ensured that both NPCs play important roles in both quests.

First Quest

The first quest is assigned to the player by John. He needs help learning the founding year of the Faculty Of Informatics at Masaryk University.

John recalls that the information is written on a blackboard in lecture room D1, but he does not know how to locate the room and is too exhausted to search for it himself. He asks the player for assistance.

Helene, the receptionist, can navigate the player to D1 if needed. While she does not know the founding year herself, she also remembers that the information is in D1 and can provide directions on how to get there. This serves as a backup in case John does not share this information himself, and the player does not think to ask.

The quest is fulfilled when the player relays the founding year to John. John then thanks for the answer, which provides clear feedback that the first quest is over. This quest is designed to familiarize the player with the faculty's layout, teach them the location of the essential lecture rooms, and teach them a bit of history about the faculty. While fulfilling the quest, the player can choose to either communicate with Helene and ask for her help or explore and try to figure the year out by themselves.

Second Quest

The second quest showcases the possibility of both NPCs assigning quests, as well as different ways of fulfilling the quest itself. This time, Helene assigns the quest. She is looking for an international student in the Computer Games Development program who would be willing to write an article about their experiences in Brno, the faculty, and their studies. The player is directed to John, who fits the description.

When approached, John expresses hesitation in writing the article. The player's task is to persuade him. The quest can be completed in two different ways: either John agrees to help and informs the player or the player reports back to Helene, stating that John will write the article. Both outcomes demonstrate the quest system's flexibility and NPCs' interactive potential.

Upon completion, the player receives feedback, such as a congratulatory message, marking the end of the second quest as well as the end of the game. At this point, the player has the option to either exit the game or remain in the environment to continue freely conversing with the NPCs. This allows the player to explore the faculty and the characters further.

5.2 Main Architecture Components

When creating a VR game, many design decisions are made with respect to one of two primary setups: PCVR or Standalone. In a PCVR setup, the game is rendered and processed on an external computer, with the headset serving as a display and input device. While this setup allows for higher graphics quality, it comes at the cost of tethering the player to a PC and requiring additional resources. On the other hand, a Standalone headset is a plug-and-play system that does not require a physical connection to a PC but is more limited in resources [131]. Given that my setup already requires a powerful GPU, I have decided to design the game with a Standalone VR headset in mind to avoid the need for an additional high-performance computer and to minimize resource requirements.

This decision, along with the requirement for an easily replaceable LLM, led to the design of a separate component for LLM management: the LLM Server. This server manages all LLM-related operations on a GPU-equipped computer, with the headset functioning as a client, sending prompts and receiving responses.

This section further explores three core components of the game, namely the LLM Server, the Game Lobby, and the Game World. It also examines how these components interact in order to facilitate player-NPC communication. Additional services such as speech-to-text and text-to-speech are essential for voice communication with NPCs, though they fall outside the scope of this thesis and will be implemented using external tools.

5.2.1 Game Lobby

The first and most simple component is the Game Lobby. It is designed as the starting point for the player, allowing them to familiarize themselves with the VR environment, practice movement, and get accustomed to game controls. It also functions as a checkpoint, ensuring that all external systems, namely the LLM Server, are up and running.

Text panels within the Lobby provide meaningful instructions on the controls and the overall story of the game. Additionally, the Lobby features a static NPC, not powered by an LLM, to help the player adapt to the game's dialogue system and practice the interaction mechanics.

Once ready, the player should be able to seamlessly transition to the game itself. This ensures that the player has a smooth and intuitive introduction to the game mechanics before progressing further.

5.2.2 Game World

The Game World is a component where the main gameplay takes place. Set within the Faculty of Informatics, it features the previously designed NPCs John and Helene, who facilitate the quests. Additionally, multiple dynamic text elements are designed to ensure that the player knows what quest is active at any given time. The environment is crafted to encourage exploration and interaction, aligning with the game's educational and narrative goals. After completing both quests, the player can choose to continue exploring or reset the game to start anew. The reset option is available throughout the entire game, allowing the player to return to the Lobby, clear the communication history, and restart gameplay without reloading the entire game.

5.2.3 LLM Server

The final core design component is a server dedicated to managing LLM operations. This server provides two primary endpoints to facilitate functionality. The first endpoint serves as a simple readiness check. Since initializing and setting up the LLM for player interactions is time-consuming, this endpoint allows the client, specifically the Game Lobby, to periodically ping the server via a simple **GET** request to verify its availability. The endpoint returns true once the server is ready, confirming that the LLM is operational. The second endpoint is designed to handle player-NPC interactions. This endpoint accepts player prompts, called from the Game World, through **POST** requests and returns the corresponding LLM-generated response along with other necessary data.

Due to hardware limitations, both NPCs in the game will use the same open-source LLM running on one GPU. While running separate LLMs simultaneously on one GPU is technically possible, doing so significantly reduces the available memory and degrades performance, leading to slower inference times. To ensure future flexibility, the

system is designed in such a way that the LLM can be easily replaced without requiring substantial rework.

The shared model approach imposes certain limitations on the game. First, in the standard sense, fine-tuning cannot be used to adapt the NPCs. Although a single fine-tuned model could be used for both NPCs (e.g., to standardize response lengths or format), loading separate fine-tuned models for each character is not possible. Second, responses from both NPCs cannot be processed simultaneously. As a result, the game is intentionally designed to allow interaction with only one NPC at a time. Therefore, a player must wait for one NPC to complete its response before engaging with the other. Lastly, each NPC's conversation history must be stored and managed separately, which will be done in the Game World component. This design enables the server to operate as an independent entity, supporting any number of NPCs without requiring modifications to its structure.

5.2.4 Player – NPC Communication

This subsection describes the designed communication workflow between the game components during player-NPC interactions, which form the core of the game experience. The player first needs to provide their input to the NPC. This can be done by activating the NPC's *listening mode*, speaking their query, and then deactivating the *listening mode*. Initially, I considered an alternative design where NPC listening would activate automatically when the player spoke within their proximity and deactivate after a period of silence to process the prompt. However, this approach risked frustration. Brief pauses in player speech could prematurely end the NPC *listening mode*. On the other hand, requiring longer silence before starting the processing could disrupt immersion, especially when combined with the LLM response generation time.

Once the player has deactivated the *listening mode*, the speech translation can begin. The generated user prompt and any previous conversation history will then be sent to the LLM Server. The server will generate a response, which needs to be subsequently converted back into speech using the TTS service and presented to the player.

During the design process, it was also important to keep in mind the possible errors that may arise. The server response generation

can fail only due to a technical error, such as the headset or server losing connection during the game. While the Game Lobby minimizes the likelihood of such failures, they cannot be entirely eliminated. In these cases, immersion may inevitably be disrupted, requiring the game to notify the player with an appropriate message about the issue. Similarly, errors in the TTS service are typically technical and would likewise necessitate an interruption to inform the player.

On the other hand, the STT service is more susceptible to player-related errors. For example, a player might speak too softly or fail to speak altogether, resulting in an empty prompt sent to the server and a confusing response from the NPC. To maintain immersion, I designed a simple error-handling mechanism that allows the NPC to address such issues while remaining in character. Instead of presenting a traditional error message, the NPC might respond by saying something like, *"I am sorry, I did not catch that. Could you repeat it for me?"*. This approach ensures that errors are communicated effectively to the player without breaking the flow of the game.

6 Implementation

This chapter outlines the implementation of Quest for Information. The first section introduces the technologies and packages utilized throughout the project. The next section delves into the implementation of the LLM Server. The following sections focus on implementing key game components, including the VR Interactions, the Lobby, the Game Scene, and the NPCs. The final section discusses Cross-component Processes such as Quest Completion, NPC Memory Management, and Error Handling, which require seamless cooperation between the server and the game.

6.1 Used Technology

This section details the technologies utilized in developing the final game. The Quest for Information is divided into two main parts: the aforementioned Python-based LLM Server and the game client developed in Unity. The following subsections describe the technologies used in each part in greater detail. A shared technology between both is GitHub [105], which was used for version control and repository management. The Quest For Information is built and tested on a standalone Meta Quest 2 virtual reality headset [132].

6.1.1 LLM Server

The LLM Server manages LLM loading, inference, and embeddings. The server is implemented in Python, which was chosen for its robust ecosystem of libraries and tools for working with large language models. Flask [133], a lightweight web framework, is used to build the server to create RESTful endpoints. This allows for efficient communication between the server and the game client.

The server leverages the same Python libraries for LLM inference as detailed in Section 4.1. Key libraries include Transformers, torch, accelerate, huggingface_hub, and scikit-learn. Together they facilitate loading the LLM, optimizing its performance on GPU, and generating embeddings for tasks such as similarity search. The server runs on the aforementioned NVIDIA GeForce RTX 4090 GPU with CUDA

support, ensuring fast and efficient model inference. The specific models include the Llama 3.2 3B Instruct and all-MiniLM-L6-v2 for generating embeddings, which were discussed previously.

The server is containerized using Docker [134], a platform for creating, deploying, and running applications within containers. Packaging the server in a Docker container simplifies the installation of dependencies, ensures compatibility across different architectures, and allows the server to be launched with a single docker-compose command. This approach enhances portability and ease of use.

6.1.2 Unity

The second component of the final application is the game client, which is developed using Unity [135], a widely used cross-platform game engine. Unity supports scripting in C#, which was utilized to implement custom functionality for the game.

Unity natively supports VR development via the XR Interaction Toolkit [136, 137], a high-level, component-based system built to enable VR interactions. This package simplifies the development process by providing tools for implementing interaction mechanics. The system revolves around Interactor and Interactable components, managed by an Interaction Manager. The XR Interaction Toolkit was used to configure the player object, integrating VR controllers and a VR-compatible camera setup. While the game includes only one grabbable or movable object, the interaction system enables the player to navigate the scene and initiate conversations with NPCs. In these interactions, the player's controllers act as Interactors, and the NPCs are treated as Interactables.

The game also leverages several external assets to enhance functionality and streamline development. The most important is the Meta Voice SDK [138] package, which provides speech-to-text and text-to-speech services, enabling interaction with the NPCs. It uses the free online Wit.AI [139] service and offers seamless Unity integration. Another example is the FXAA (Fast Approximate Anti-Aliasing) [140] asset, which enhances rendering quality by improving anti-aliasing, ensuring smoother visuals. I also utilized additional external resources for the character visuals [141, 142]. Further resources are linked both in the code and in the corresponding chapters.

6.2 LLM Server

The LLM Server is a containerized application built using Docker. The server can be launched by navigating to the repository and executing the `docker compose up` command. This command first installs all specified Python packages and then executes the necessary scripts. Before becoming operational, the server runs several initialization steps. First, it downloads and initializes the LLM and the embedding model. Then it precomputes the embeddings for use in quest management, which will be analyzed in more detail in subsection 6.7.1.

As per design, the server provides two endpoints for communication. The first is the **status** endpoint, which responds to simple **GET** requests. This endpoint does not require any input and only returns true to confirm the server is active. The second is the **chat** endpoint, which accepts **POST** requests with a JSON body. This endpoint enables interaction with the LLM by processing the user prompt and returning an LLM-generated response. It is reusable for all current and future NPCs in Quest for Information.

The **chat** endpoint expects the following fields in the JSON request:

- **Conversation History:** A list of messages representing the player's and NPC's conversation history. Each message consists of two attributes:
 - **role:** Specifies the role of the author of the message as either *System*, *User*, or *Assistant*. As per LLM requirements, *User* and *Assistant* roles must alternate, while *System* messages can appear anywhere in the sequence. The history must end with the player's message (role *User*), which contains the user prompt that the LLM will answer.
 - **text:** The content of the message.
- **Chatbot Name:** A unique identifier for the NPC associated with the interaction. This identifier does not need to match the NPC's real name but must be consistent between the game client and the server. Its significance is further discussed in subsection 6.7.1.
- **Quest:** An integer value representing the currently active quest. Once again, its purpose is described in subsection 6.7.1.

- **Player Quest Completion:** A boolean indicating whether quest completion should be triggered by the player's input (true) or by the NPC's response (false). This field is also further explained in subsection 6.7.1.

The server processes the input, evaluates quest completion, generates the LLM response, and returns a JSON object with the following fields to the client:

- **Response:** A string containing the NPC's reply.
- **Quest Completion:** A boolean indicating whether the current quest has been successfully completed (true) or not (false).
- **Needs Refresh:** A boolean noting whether the conversation has reached a length that risks exceeding the LLM token limit. If set to true, it triggers the in-game memory management system, further discussed in subsection 6.7.2.

Additionally, the LLM used by the server can be easily replaced. The server uses standardized libraries compatible with several open-source models available from Hugging Face, such as Llama, Mistral, or Gemma. While proprietary LLMs, like GPT, would require some code changes, the overall process remains straightforward. This design ensures flexibility in updating or switching models without needing significant changes to the server's structure, allowing the system to easily adapt to future advancements in LLM technology.

6.3 VR Interactions

The player object is built on top of the XR_Rig from the XR_Interaction Toolkit, which controls the player's perspective and view within the game. Unity also offers a ready-to-use system for VR controllers, along with their in-game models. The system simplifies the process of mapping in-game actions to the physical buttons of the Oculus Quest 2 controllers. A layout of these controllers and their button assignments is provided in Figure 6.1.

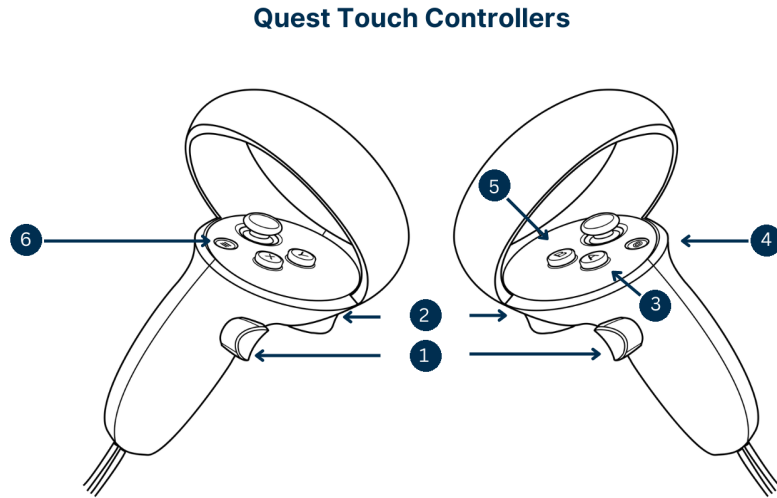


Figure 6.1: A model of Oculus Quest 2 controllers [143] with the following buttons: 1 – grip button, 2 – trigger button, 3 – primary button, 4 – oculus button, 5 – secondary button, 6 – menu button.

Both controllers are equipped with a straight ray that enables interactions throughout the game. This ray allows the player to receive visual feedback on interactable and non-interactable elements, as well as interact with the user interface (UI) within the Lobby and the Game World. In addition, each controller is equipped with a custom teleportation ray that enables movement. By pressing and holding the primary button on either controller, the straight interaction ray transitions to a curved teleportation ray. The player can then move their controller to select a teleport destination. The ray appears blue for valid teleportation areas and red for invalid ones. Once the button is released, the player is teleported to the selected destination. Both rays are visible in Figure 6.2. Additionally, the player can use the thumbsticks on both controllers to rotate their view and orient themselves within the environment.

The remaining controller buttons are mapped to various interactions across both the Game Lobby and Game World scenes. The grip button is mapped onto the grab action, which is used exclusively in the Lobby for manipulating a movable VR keyboard. Across both scenes,

the trigger button functions as a select tool used for navigating UI elements. It is also central to NPC interaction; to engage, the player must point their interaction ray at the character, press and hold the trigger while speaking their query, and release it to receive a response. Furthermore, the menu button on the left controller is dedicated to opening a small UI panel that panel allows the player to exit the game and return to the Lobby without the need to reload the application.

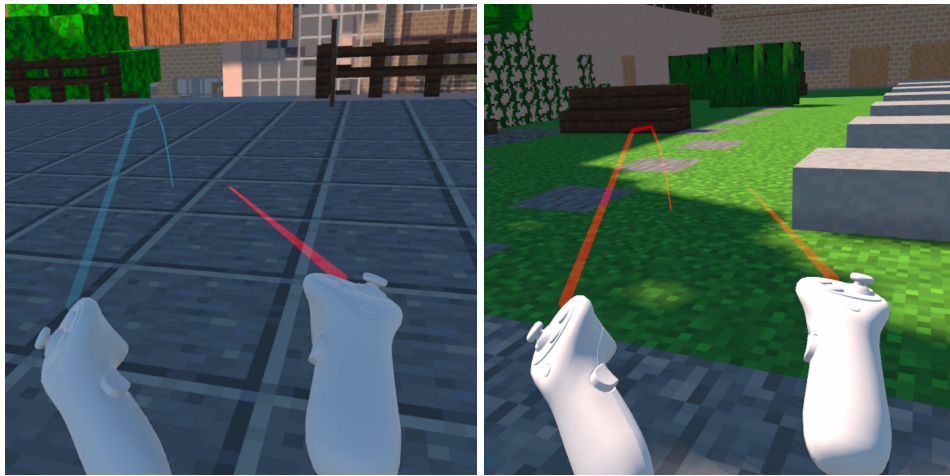


Figure 6.2: The curved teleport and straight interaction ray. A valid teletransportation ray is displayed on the left, and an invalid teletransportation is displayed on the right.

6.4 Game Lobby

The Lobby (as seen in Figure 6.3) is designed as a waiting room for the server, as well as a short introduction to the game. It is constructed using free-to-use 3D models [144]. Additionally, it features several interactable text panels that introduce the player to the controls and the game storyline.

To not overwhelm the player, the UI elements are presented sequentially. The first screen welcomes the player and introduces the game. The following are multiple panels explaining the game's controls. The next UI explains the objectives and quests, outlining the player's goals.

Finally, the last panel lets the player input the server's IP address and join the game using a simple button. Initially, the **Join Game** button is inactive, displaying a message that the server is unavailable. When a valid IP is entered, and the server becomes reachable, the button becomes active, allowing the player to proceed. Additionally, each UI screen offers **Previous** and **Next** buttons to ensure the player can revisit any information they might have missed during the first readthrough. Some of the panels can be seen in Figure 6.4.

A VR keyboard asset from Black Whale Studio [145] is used to enter the IP address of the server. The player types by pointing to the desired keyboard character and pressing the controller's trigger button. The keyboard is spawned directly in front of the player when they select the IP address text field for input. The keyboard itself is also movable, allowing the player to reposition it if necessary. It can also be observed in Figure 6.4.

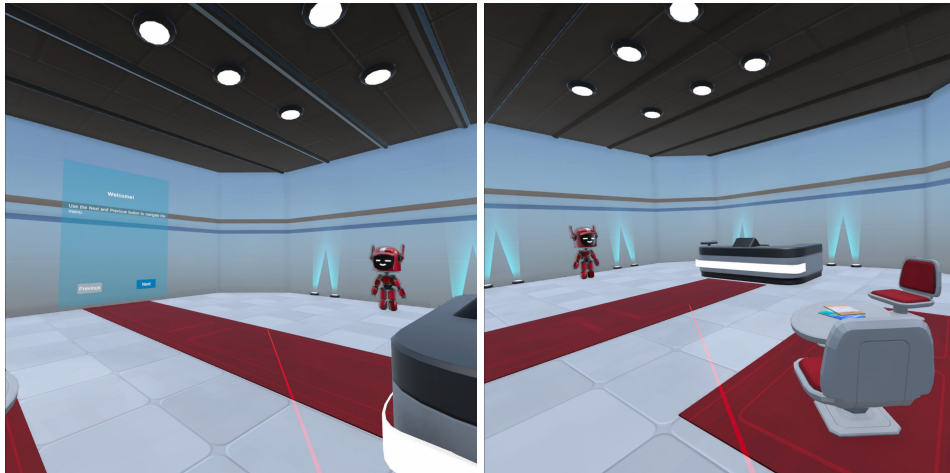


Figure 6.3: Screenshots from the Lobby room.

The dynamic **Join Game** button is controlled by a background script that periodically pings the server URL on the designated **status** endpoint. The button is activated as soon as the script receives a valid response from the server, and the player is allowed to join the game. If the server becomes unreachable or the player further modifies the IP address, the button reverts to its inactive state.



Figure 6.4: The introductory text panels. The top left image explains the game to the player. The top right image explains the NPC interaction controls. The bottom images show the panel connecting the player to the game alongside the VR keyboard (with an unreachable server displayed on the left and an available server on the right).

The player can also freely navigate the Lobby to practice the teleportation and snap rotation. A small robot (visible in Figure 6.5) allows the player to test NPC interaction mechanics by speaking to it. Unlike the in-game NPCs, the robot cannot be used for traditional communication. Instead, it simply repeats the player input using the

speech-to-text system to translate the query and the text-to-speech system to vocalize it. This interaction helps the player understand the limitations of the STT functionality and familiarize themselves with communication mechanics before engaging with characters in the game. It also serves to verify the game's microphone access and ensures that the STT service is functioning properly.



Figure 6.5: Robot in the Lobby repeating the players speech: *"Hello, who are you?"*.

6.5 Game World

Once the player clears the Lobby and establishes a correct connection to the server, they can move on to the game itself. The player starts in front of the Faculty of Informatics. The detailed model of the faculty, including its textures, was created by students of the Faculty of Informatics and provided to me by my supervisor [146]. The in-game view of the faculty can be seen in Figure 6.6.

Multiple teleportation areas are placed throughout the faculty to allow restricted player movement. These areas include the path leading up to the main entrance, the faculty lobby, the inner courtyard on both levels, and the first and second floors of the D block, which houses lecture rooms D1 and D2.

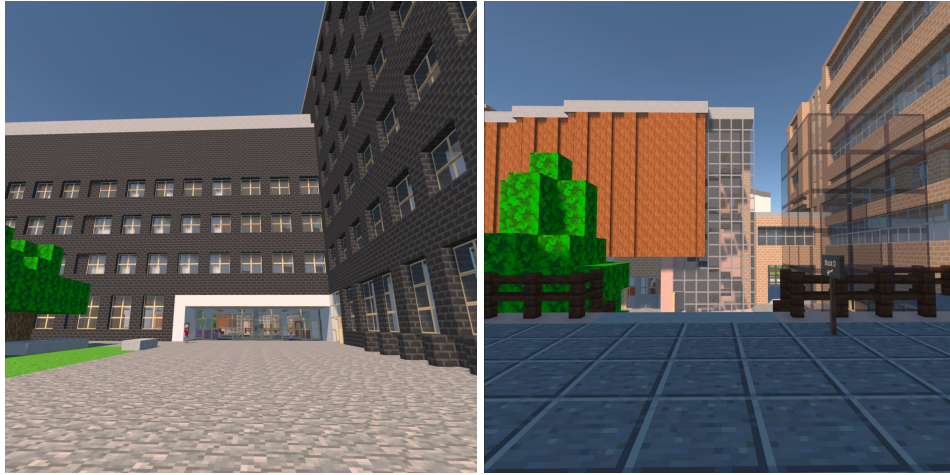


Figure 6.6: Screenshots showing the in-game model of the Faculty of Informatics, Masaryk University.

Several guiding text elements are put up around the faculty. The first, recapitulating all VR controls, is found next to the player's starting point. It serves as a helpful reminder in case a player has forgotten how to navigate the game. A quest board is located near the main entrance, outlining the player's goal. Furthermore, a small text indicator is placed over the player's left arm, also with small information about the first quest: *"Talk to John"*. Once this first task is completed, the indicator updates to display the second quest: *"Talk to Helene"*. Some of these in-game text elements can be observed in Figure 6.9. The core gameplay, including both quests (described in more detail in Section 5.1.2)), takes place in the faculty. Several additional elements that help guide the player to lecture room D1 during the first quest are shown in Figure 6.7.

Upon completing the first quest, the quest indicators, including the text on the quest board and on the player's hand, are updated to display guidance for the second quest. A new poster, authored by Helene, appears in the faculty lobby, stating her need for help with an article. The player should then speak with NPC Helene, who assigns the second quest – finding a student enrolled in the Computer Game Development program to assist her in writing an article for the school website. This task requires the player to talk to John again

and persuade him to help Helene. The poster, alongside the modified quest board, is also visible in Figure 6.9.



Figure 6.7: The image on the left shows the labeled lecture room D1 with the founding year written on the blackboard. The image on the right shows the small arrow pointing to the D block.

When both quests are completed, a final text screen (visible in Figure 6.8) appears in front of the player, notifying them of their success. This UI also provides the option to return to the Lobby or continue exploring the environment and interacting with NPCs.



Figure 6.8: The final screen is shown after the completion of both quests.



Figure 6.9: The in-game text panels. The top images show the changing quest board from the first quest (left) to the second quest (right). The bottom left image shows Helene’s poster, while the bottom right image displays an image of the VR controls.

6.6 Non-player Characters

Both NPCs in the game share the same functionalities and differ only in their personal information and visuals. To easily integrate a new NPC into the game, developers can drag and drop a prepared object into any Unity scene. The implementation details surrounding the NPCs are divided into four parts, namely the Visuals, the User interface, the technique for LLM customization used to suit specific character roles, and the implementation framework that provides communication between the player and NPCs.

6.6.1 Visuals

In order to appear more lifelike, NPCs are equipped with a tracking script that allows them to follow the player's movements. If the player moves slightly, the NPC's head rotates toward them, maintaining eye contact. If the player moves even more, the NPC also rotates its body to match the movement of the head. This dynamic tracking adds a sense of presence and responsiveness to the NPCs, enhancing the immersive experience.

Both NPCs are also provided with an outline [147] that serves as a visual indicator, signaling whether the character is available for interaction. This outline is hidden by default but becomes visible when the player hovers their interaction ray over the NPC. The outline's color changes based on interaction status: green indicates readiness for interaction, while red signifies that the NPC is currently unavailable. Both characters, along with the outline, can be seen in Figure 6.10.



Figure 6.10: The image on the left shows NPC Helene with a green outline. The image on the right shows John with no outline, and Helene is visible in the background. Both NPCs rotate to look at the player.

6.6.2 User Interface

Dynamic texts (observed in Figure 6.11) are displayed above the NPC's heads to provide the player with real-time feedback during interactions. These texts serve as intuitive guides and status indicators:

1. **Processing Indicator:** When the NPC is processing the player's input and generating a response, an animation of three dots appears. This universally recognized "loading" animation reassures the player that the NPC is working on a response.
2. **Speech Transcription:** As the player speaks to the NPC, their input is transcribed and displayed in real time using the STT system. This dynamic feedback allows the player to adjust their sentences early, ensuring greater accuracy.
3. **Interaction Restriction:** If an NPC is unavailable due to another interaction in progress, the text *"Another NPC is speaking"* appears. This text complements the red outline on the NPC, reinforcing that the player must wait.



Figure 6.11: The image on the left shows the Interaction Restriction text. The image on the right shows the speech transcription along with the processing indicator.

6.6.3 LLM Customization

This subsection outlines the approach taken to customize the Large Language Model for in-game characters. Chapter 4 introduced several methods for LLM Customization, including extended system prompts, fine-tuning, and embeddings (or RAG). Fine-tuning was not a viable option due to resource limitations. Creating a separate fine-tuned LLM for each character would lead to a performance-heavy solution that was not scalable. While a single fine-tuned LLM could be used to, for example, shorten responses, it would create more complications when replacing the LLM with a stronger model in the future and, therefore, was deemed unnecessary.

Since the game simulates a realistic environment rather than a purely fictional one, there was no need to heavily restrict the LLM's knowledge. Section 4.4.4 demonstrated that both RAG and the extended system prompt produced similar results, making either suitable for customization. Ultimately, the extended system prompt was chosen due to the priority placed on minimizing context mistakes over better response adaptation. While embeddings were not used to influence character responses directly, they played a key role in determining quest completion, which is discussed further in Section 6.7.1.

NPC documents

The extended system prompt is dynamically generated using a collection of user-written NPC documents. Each of these documents serves a distinct role in defining the NPCs' behavior and world context:

- **World File:** A file shared among all game NPCs containing general information about the game world. It informs the NPCs that they are at the Faculty of Informatics in Brno while adding details about the faculty layout and other shared information.
- **Personal File:** This file contains unique attributes for each individual NPC. It includes essential information such as their name, age, backstory, personality traits, and interests.
- **State File:** This file contains the current in-game state of the NPC, such as their location or specific situational detail.

- **Quest Files:** This set of files contains information relevant to specific quests. Every NPC needs to have a file for each quest. If an NPC does not participate in a particular quest, their corresponding file can remain empty or, preferably, state that the NPC *"does not need anything at the given time"*. These files help the system tailor interactions based on quest progression. Their effect on the quests will be further explored in section 6.7.1.

The system prompt is constructed by concatenating text from these documents alongside generic instructions for the LLM. Examples of such instructions include statements like *"Act as if you are the character..."* or *"Avoid narrating the scene"*. This document structure provides flexibility, allowing developers to add or modify NPC details and behaviors without needing to alter the LLM configuration. Additionally, it simplifies the creation of new quests and NPCs, offering a dynamic and scalable approach to expanding the system.

6.6.4 NPC Logic

The functionality of the NPCs relies on a logical framework composed of four main components: the NPC Manager, Speech-To-Text, LLM Client, and Text-To-Speech modules.

NPC Manager

The NPC Manager acts as the central hub for communication, coordinating interactions between the STT, LLM Client, and TTS components. When a player speaks to an NPC, the Manager triggers the STT service to process the voice input and receives either the translated speech or an error message. In the event of an error, the Manager generates an alternative prompt for the LLM (see Section 6.7.3 for details on error handling). The prompt is sent to the LLM Client for response generation and then passed to the TTS service for audio output. Once the audio is finished playing, the interaction concludes, and the Manager resets all NPCs, making them ready for the next interaction.

STT

The STT service leverages the Voice App Experience from the free-to-use Meta VoiceSDK Package. A dedicated script handles the start and stop functions for recording the player's speech. The STT service might occasionally fail if the player speaks too quietly or does not speak for a longer period of time. All transcription results, whether successful or not, are sent back to the NPC Manager for further handling.

LLM Client

The LLM Client is the intermediary between the NPC and the LLM Server. The LLM Client manages all data related to prompting the LLM, namely the generation of the system prompt and the conversation history between the player and the specific character.

The LLM Client receives the player's translated speech from the NPC Manager and packages it along with the conversation history and other relevant data into a JSON request. This request is then sent to the LLM server. Upon receiving a response, it decodes the output, updates the conversation history accordingly, and sends the response back to the NPC Manager.

Conversation history is initiated with a base system prompt, followed by specific prompts related to player-NPC interactions or game-play events, such as a change in quests. The system prompt is generated by concatenating the correct documents, as specified in Subsection 6.6.3. As interactions progress, the conversation history grows, ensuring that the NPC maintains the recollection of all interactions throughout the game.

TTS

Finally, the TTS service translates the NPC's text response into spoken dialogue. It is implemented using the TTSWitService and TTSSpeaker modules from the VoiceSDK by Meta. TTSSpeaker allows for voice customization, enabling developers to select voices that best match each character's persona. Unfortunately, the TTSWitService is limited to 280 characters per response. In order to address this, an additional script was developed to logically split longer responses into smaller, manageable chunks, which are then spoken sequentially.

6.7 Cross-component Processes

This section further explains processes that involve close cooperation across multiple components of the system. These include quest management, NPC memory management, and error handling during speech-to-text (STT) interactions.

6.7.1 Quest Management

Quest management is used to assign the correct quest to the player and learn when the quest is completed. It requires collaboration between the LLM Server, NPCs, and a central Game Manager. The Game Manager keeps track of the active quest, references to all NPC Managers, and connections to the dynamic quest texts.

The process of quest management can be broken down into three key steps: Initialization, Quest Completion, and Story Progression. (1) Initialization, done at the beginning of the game, prepares the data required for managing all game quests. (2) Quest Completion handles the actions of both the player and the NPCs that can lead to a quest being completed. Quests can be completed either through player queries or NPC responses. (3) Story Progression is triggered once a quest is finished. During Story Progression, the Game Manager updates the game state and informs all relevant components.

(1) Initialization During the initialization of the LLM Server, quest embeddings are prepared using stored character files. For each quest-character pair, the server reads two text files: Positive and Negative. These files are optional, meaning if they are missing or empty, they will not be used when generating NPC responses. Furthermore, if an NPC lacks a Positive file for a particular quest, it indicates that the NPC cannot be used to complete that specific quest.

- **Positive Files:** These contain sentences that represent possible queries that can be said to complete the quest. These sentences may originate from either the player or the NPC, depending on the quest's structure. For example, in Quest 1 for NPC John (which is completed by the player), the Positive file might include phrases like:

- *"The year is 1994."*
- *"I went to D1, and I found out that the year is 1994."*
- *"I think the faculty was founded in 1994."*
- **Negative Files:** These contain questions the player might ask the NPC when trying to learn forbidden information. These sentences are used (in addition to the system prompt) to anticipate the player's intent and prevent the NPC from disclosing details it should not share. For instance, in Quest 1 for John, the Negative file ensures that John cannot assist the player in finding lecture room D1. It includes lines such as:
 - *"Do you know where lecture room D1 is?"*
 - *"How do I get to D1?"*

Once these files are loaded, their content and precomputed embedding values are stored in a nested dictionary structure for efficient access. When the server is ready and the player reaches the game, each NPC is initialized with a conversation history containing the main system prompt setting up the character, and another system prompt initializes the first quest using the relevant Quest Document (see Section 6.6.3). For instance, John's first quest system prompt includes the following instructions: *"You are looking for the founding year of the Faculty of Informatics. You heard that it was written on the blackboard in lecture room D1, but you are not sure where it is located and how to get there. You are too tired to go yourself..."*.

I generated all of the Positive and Negative files with help from ChatGPT. To start, I focused on identifying the possible sentences that players or NPCs might say when completing each quest. Since every player is different, I created multiple variants of each response by prompting ChatGPT to generate similar queries. Generating the NPCs' responses was a bit more complex. For example, predicting what an NPC John might say when he finally agrees to write the article (see Section 5.1.2 for additional quest detail) proved challenging, as NPCs tend to be more talkative than players.

(2) Quest Completion Once a player interacts with an NPC, the system determines whether the current quest can be completed based on the

character Quest file (see Section 6.6.3). The first line of each character Quest file specifies who is responsible for completing the quest: Player (the quest can be completed through a player query to this NPC), NPC (the response of this NPC can complete the quest), or No one (the quest cannot be completed by interaction with this NPC).

For instance, in the case of the Quest files for NPC John, the first quest can be completed by the player speaking a sentence, "The founding year of the faculty is 1994". The NPC John can complete the second quest by saying, *"Alright, you have persuaded me to write the article"*.

- **Player Completion:** The server computes the embedding of the player's query and compares it against the embeddings of the NPC's corresponding Positive file. If a match is found with sufficient similarity, the server considers the quest fulfilled. The prompt is modified to inform the NPC of quest completion, e.g., by adding: *"The user told you the information you were looking for"*.

On the other hand, if no match is found, the query gets further verified against the Negative embeddings. If a match is found here, the user prompt is modified to include additional text – *"You do not know this information."* – to ensure no forbidden information is leaked to the player.

If no match is found in either file, the player's prompt is updated to reflect the fact by adding *"If you feel like the player gave you the response you are expecting, it is not true"*. This line ensures that the NPC does not generate a response containing false feedback on the quest status.

Finally, the LLM response is generated and returned to the NPC, along with the information on the quest completion.

- **NPC Completion:** In this flow, the player's prompt is first checked against the Negative embeddings. If a match is found, the prompt is modified the same way as before. Afterward, the LLM is used to generate a response to the original player query. This response might contain the information that completes the quest.

This LLM response is then embedded and checked against the embeddings of the Positive file. Here, the similarity threshold is slightly lower since LLM-generated text can be less predictable.

If a match is identified, the system expects that the LLM said something that indicates the completion of the quest. Finally, the server returns the already generated response, along with the information on the quest status.

(3) Story Progression When the NPC receives information about quest completion, it reports to the Game Manager, which assigns a new quest, updates the relevant UI elements, and informs all NPCs that they need to add a new system prompt to their conversation history that reflects the new quest.

6.7.2 NPC Memory Management

Memory management is one of the known challenges in LLM-based chatbots due to token limitations (see Section 2.4.4). While advanced solutions exist, I opted for the more straightforward approach: periodic summarization. During every interaction between the player and an NPC, the LLM Server calculates the total number of tokens used in the conversation so far. Once this number reaches over 75% of the allowed maximum tokens, the server informs the NPC that it needs to regenerate its memory soon.

To regenerate memory, the NPC then sends an out-of-context prompt to the LLM Server, requesting a summary of the entire conversation thus far. The resulting summary becomes the new starting point for the NPC's memory, which ensures that essential information is retained while freeing up space for future interactions.

Timing the summarization correctly is critical to avoid disrupting the player's experience. Summarization cannot be triggered while the player is actively interacting with an NPC or preparing to do so, as this would cause a response delay and break immersion. To determine the correct timing, a Game Manager periodically checks the state of all NPCs in the scene. If any NPC requires memory regeneration, the Game Manager checks whether the player is currently interacting with an NPC or hovering their controller over one (indicating potential interaction). The summarization request is postponed if an interaction is in progress or likely to occur. Otherwise, the Game Manager grants permission for the memory regeneration.

6.7.3 Error Handling

Two primary services can encounter failures when interacting with NPCs: the STT and the LLM Client. Failures in the LLM Client are critical because they typically indicate either server-side issues or a loss of connection within the user's headset. In such cases, the player is forced to resolve the technical problem manually, resulting in a break of immersion that cannot be avoided.

On the other hand, errors in the STT system are more common and are generally due to player behavior. These errors can occur when the STT fails to recognize speech because the player's voice is too quiet (see Subsection 6.6.4). Such issues can be addressed through in-game feedback without disrupting immersion.

When an STT error occurs, the LLM ensures the issue is handled smoothly. If the STT system fails, a custom prompt is created, urging the LLM to explain the problem to the player in a way that stays true to the NPC's character. This special request is sent to the LLM Server with no prior conversation history other than the original system prompt. The server responds with a personalized error message that the NPC then relays to the player. This interaction is not stored in the conversation history, so it does not disrupt future dialogue. This method takes advantage of the LLM's creativity to deliver helpful and engaging error messages, all while keeping the player immersed.

7 Evaluation and Future Work

This chapter focuses on the evaluation of Quest for Information and outlines areas for future improvement. The first section presents results from a short user test, followed by the second section, which evaluates how the solution meets the set requirements. Finally, the third section explores possible directions for future work.

7.1 User Testing

Quest for Information was tested by three participants: two experienced VR users and developers and the head of the public relations and marketing division at the Faculty of Informatics, Masaryk University. The testers' varying levels of experience with VR provided valuable insights into the concept of LLM-powered NPCs and the overall experience.

Feedback from the head of public relations and marketing division was overwhelmingly positive. Since their prior experience with VR was limited, the focus was placed on the educative potential of the game. They praised the game-based learning approach, particularly the ability to familiarize players with the faculty's layout while learning information, such as the faculty's founding year. They viewed Quest for Information as an already useful educational game while also appreciating the potential of expanding the game world and the game characters in the future. For potential future expansion, they suggested the addition of a study department assistant NPC. This assistant could provide prospective students as well as first-year students with helpful information, such as admission requirements or general tips. Additionally, the use of VR was seen as a key strength, as it enhanced engagement in exploring the faculty environment.

The two VR-experienced testers focused more on the game's technical aspects and interactions with the NPCs. Both players preferred to interact with NPCs rather than explore the environment alone, and therefore, they showed a high level of engagement in the interactions. When observing another player playing the game, they noted that the LLM inference times appeared slower than expected. However, when playing the game themselves, the user experience was not affected by

the suboptimal performance because they had to focus on verifying their spoken prompt to validate the correctness of the transcription. Additionally, the loading animation reassured them that the NPC was actively working on a response.

Additionally, some users identified issues with the STT tool. The transcription service occasionally timed out and provided only partial results when players paused their speech for too long. Voice recognition errors also caused occasional confusion. This issue was most noticeable during the second quest, where John was originally intended to be an Erasmus student. However, Erasmus was repeatedly unrecognized, leading to a redesign of the quest. In the final version, John's background was changed to that of a student in the Visual Informatics program. While the transcription errors occasionally disrupted gameplay, they caused fewer problems than expected, as players could always verify their input using the displayed text.

Both VR testers, who regularly work with LLMs, recognized some common limitations of LLM-powered chatbots, such as occasional answer repetition or rigid responses. Leveraging their LLM knowledge, they experimented with the NPCs to explore edge cases and intentionally "break" the gameplay. This approach resulted in humorous situations, which in turn further encouraged NPC interaction. Most notable issues occurred when players asked the NPCs to perform physical actions, such as walking somewhere or talking to someone else. For example, while in the first quest, John correctly refused to search for the lecture room himself, stating that he was too tired, in the second quest, Helene could be persuaded into believing she should persuade John herself, which was technically impossible. One tester even encountered issues with action narration, which did not occur in any of the other playthroughs. These findings suggest room for improvements, either in strengthening the constraints on NPC behavior (e.g., John refusing to move due to exhaustion) or enabling physical actions for the characters.

Furthermore, the NPCs were found to be unpredictable at times, which was particularly noticeable during repeated playthroughs. While this unpredictability enhances replayability by providing players with varied experiences, it also introduces several challenges. They were mostly seen in the second quest. There, players occasionally struggled to convince John and reported sometimes getting stuck in conversa-

tional loops — either believing they had successfully persuaded him or assuming they were on the wrong path to quest completion.

Overall, the feedback was highly positive. Players praised the VR environment and the engaging NPCs. Players always understood their objectives. Even when they attempted to overcome them by untraditional means, such as confusing the NPCs, this resulted in deepening their interest in the interactions, which was the desired goal. The system successfully maintained immersion with quick responses and visual elements. However, the limitations of using a small LLM occasionally became apparent, which can be expected when using a model with a smaller number of parameters. It is important to note that while these insights are valuable, more extensive user testing and continued development are needed to draw any definite conclusion. Conducting such tests is, however, beyond the scope of this thesis.

7.2 Compliance with Requirements

In this section, I further discuss whether Quest for Information meets all requirements proposed in Chapter 3. It is also important to note that while the initial testing shows that the system satisfies the set requirements, more extensive user testing is needed to gather data on both player and developer experiences to fully evaluate game compliance. This testing should be addressed in future works, as it is beyond the scope of this thesis.

The first requirement is communication with NPCs. Players are able to communicate with two NPCs that were customized to understand the environment. Testing shows that the LLM responses were relevant, although some hallucinations were present due to the limited size of the used LLM.

The second requirement focuses on the presence of game-based learning. The game's educational potential lies in encouraging the players to familiarize themselves with the layout of the faculty and remember the founding year.

The potential use of Quest for Information for events such as Faculty Open Days is discussed in more detail in Section 7.1. I chose to develop Quest for Information in virtual reality in order to enhance immersion. Additionally, immersion is improved by high levels of

engagement shown during testing since players were keen on actively interacting with NPCs rather than exploring. However, since the sense of immersion may vary among players, further testing is needed to fully evaluate the compliance with this requirement.

The requirement to use open-source tools is satisfied in both the used LLM and other features, such as the embedding model and the additional Unity libraries. An exception was placed on the STT and TTS services. Although they are also open-source, they currently run on external, proprietary servers.

Furthermore, the requirement to align the game with the Faculty Open-Days Event and incorporate game-based learning principles was achieved and evaluated during the user testing.

The requirement for an easily replaceable LLM is met by using the transformers library in the LLM Server implementation (see Section 6.1.1). The library provides an abstraction over multiple models, simplifying the switch to a different model supported by Hugging Face. For example, changing the current Llama model to a stronger version requires only a change in the definition of the model path and the maximum amount of tokens allowed by the model. While integrating proprietary models, such as GPT, would require additional modifications, the system remains adaptable to future advancements in LLM technology.

Finally, Quest for Information is designed with the requirement for an easily expandable game world in mind. Adding new NPCs involves a straightforward process: dragging a pre-prepared character object into the scene, modifying the visual assets (such as the textures and a correct name tag), and creating NPC-specific behavior documents. Similarly, adding new quests requires only writing new quest documents and preparing embedding files. Therefore, no changes to the code itself are necessary. It is important to note that while I performed additional testing and found that these methods worked with no further complications, they were not tested on any external user or developers.

Overall, Quest for Information provides a strong foundation for further expansion. From the requirements placed on the LLM itself, the game integrates an open-source LLM alongside an open-source embedding model. In terms of LLM speed, players reported comfortable inference times during gameplay, though, once again, additional

testing results might be needed. Lastly, the LLM’s influence over the game can be observed throughout the quest system, which encourages meaningful interactions.

7.3 Future Work

As LLM technology continues to evolve (see Section 2.4.4), the game can also be adapted to leverage newer and stronger models. This includes advancements not only in LLMs but also in STT and TTS systems. Furthermore, running the LLM Server on more powerful hardware, multiple GPUs, or a distributed server setup could support more complex models and reduce response times, leading to higher-quality interactions.

Additional improvements can be applied to the quest completion system. This is motivated by reports from the players, who were able to confuse the LLM, particularly during the second quest. In that quest, the completion criteria were less precise, making it harder for the system to determine whether the quest was completed or not. In the future, quest completion could be evaluated directly by the LLM. Instead of comparing prompts to embeddings, the server could first ask the LLM whether the quest was completed and then prompt it again for the character’s response. While this approach was not feasible due to limited resources, advancements in hardware could make it viable without sacrificing immersion.

The game’s current memory management system also offers room for improvement. Although token limits were not an issue during the short playtime, a larger game with extended interactions could require a more robust memory system. For instance, tools like MemGPT [94] could replace simple conversation recapitulation, thus enabling more dynamic and scalable memory management. Additional future work involves replacing the STT and TTS services that currently rely on an external server.

8 Conclusion

The primary objective of this thesis was to develop a prototype for Quest for Information, an immersive educational game where players interact with LLM-driven non-player characters through open-ended questions. The NPCs play an active role in the game and are integral to the player's progress. The prototype was designed in accordance with requirements created in collaboration with my supervisor.

Quest for Information serves as a proof of concept to explore the potential of LLM-powered NPCs in educational gaming. A simple quest system was implemented, in which NPCs are responsible for assigning tasks, tracking progress, and fulfilling quests. Quests are completed through speech-based interactions initiated by the player.

Given the importance of immersion in game-based learning, the game was designed for a VR environment. Players explore the Faculty of Informatics of Masaryk University and complete learning-based quests, such as discovering the founding year of the faculty. The resulting framework is reusable in any Unity-based game, providing a foundation for future development.

In addition to the prototype, this thesis also focused on customizing LLMs for use in games, exploring techniques such as extended system prompts, fine-tuning, and Retrieval-Augmented Generation. An additional result of this work is a set of developer-friendly tools for recreating these efforts.

In the future, the game can be extended by introducing more NPCs or quests. Furthermore, as LLMs continue to evolve and GPU capabilities improve, more powerful models can be integrated into the game, further enhancing the player experience.

In this thesis, I first explored existing work on LLMs in education and gaming. I then gathered requirements for Quest for Information and analyzed available open-source LLMs and customization approaches. Based on these findings, I designed and implemented the prototype. It was then tested with three participants and evaluated against the predefined requirements. Overall, the resulting game demonstrates significant potential for use in educational contexts, such as Faculty Open Days, and provides possibilities for further expansion.

Bibliography

1. SINGH, Shubham. *CHATGPT statistics 2024 – 300 million active users (December)*. 2024. Available also from: <https://www.demandsage.com/chatgpt-statistics/>.
2. KASNECI, E.; SESSLER, K.; KUCHEMANN, S.; BANNERT, M.; DEMENTIEVA, D.; FISCHER, F.; GASSER, U.; GROH, G.; GUNNEMANN, S.; HÜLLERMEIER, E., et al. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*. 2023, vol. 103, p. 102274.
3. KICKMEIER-RUST, M.D.; ALBERT, D. Micro-adaptivity: protecting immersion in didactically adaptive digital educational games. *Journal of Computer Assisted Learning*. 2010, vol. 26, no. 2, pp. 95–105. Available from doi: <https://doi.org/10.1111/j.1365-2729.2009.00332.x>.
4. PIAGET, Jean; GATTEGNO, Dr; HODGSON, Miss F. M. *Play, dreams and imitation in childhood*. Heinemann, 1951.
5. RIEBER, Lloyd P. Seriously considering play: Designing Interactive Learning Environments based on the blending of microworlds, simulations, and games. *Educational Technology Research and Development*. 1996, vol. 44, no. 2, pp. 43–58. Available from doi: 10.1007/bf02300540.
6. VYGOTSKY, L. S. *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard University Press, 1978. Originally published in 1933.
7. AMORY, Alan. Game object model version II: A theoretical framework for educational game development. *Educational Technology Research and Development*. 2006, vol. 55, no. 1, pp. 51–77. Available from doi: 10.1007/s11423-006-9001-x.
8. SELEN TURKAY, Daniel Hoffman; KINZER, Charles K.; CHANTES, Pantiphar; VICARI, Christopher. Toward Understanding the Potential of Games for Learning: Learning Theory, Game Design Characteristics, and Situating Video Games in Classrooms. *Com-*

- puters in the Schools*. 2014, vol. 31, no. 1-2, pp. 2–22. Available from doi: 10.1080/07380569.2014.890879.
9. JAN L. PLASS, Bruce D. Homer; KINZER, Charles K. Foundations of Game-Based Learning. *Educational Psychologist*. 2015, vol. 50, no. 4, pp. 258–283. Available from doi: 10.1080/00461520.2015.1122533.
 10. UNIVERSITY OF WATERLOO CENTRE FOR TEACHING EXCELLENCE. *Gamification and Game-Based Learning*. 2023. Available also from: <https://uwaterloo.ca/centre-for-teaching-excellence/catalogs/tip-sheets/gamification-and-game-based-learning>. Accessed: 2024-11-11.
 11. DUOLINGO, INC. *Duolingo: Language Learning App*. 2024. Current as of 2024. Available also from: <https://www.duolingo.com>. Accessed: 2024-12-10.
 12. KAHOOT! AS. *Kahoot!: Game-Based Learning Platform*. 2024. Available also from: <https://kahoot.com>. Accessed: 2024-12-10.
 13. CODECADEMY, INC. *Codecademy: Interactive Coding Lessons*. 2024. Available also from: <https://www.codecademy.com>. Accessed: 2024-12-10.
 14. BURKE, Anne; TULLY, David; EDWARDS, Alison; FOUNTAINE, Lara; DAHILL-MOORE, Amanda; VALENCIA, Alexander; CULLER, Tori; SAROZA, Cas. *Mobile Scavenger Hunt*. 2012. Available also from: <https://www.lib.ncsu.edu/projects/mobile-savenger-hunt>.
 15. PHO, Annie; DINSCORE, Amanda. *Game-Based Learning: Tips and Trends*. Instructional Technologies Committee, ACRL, 2015. Available also from: <https://acrl.ala.org/IS/wp-content/uploads/2014/05/spring2015.pdf>. Accessed: 2024-11-11.
 16. HUTZLER, Armin; WAGNER, Rudolf; PIRKER, Johanna; GÜTL, Christian. MythHunter: Gamification in an Educational Location-Based Scavenger Hunt. In: BECK, Dennis; ALLISON, Colin; MORGADO, Leonel; PIRKER, Johanna; KHOSMOOD, Foaad; RICHTER, Jonathon; GÜTL, Christian (eds.). *Immersive Learning Research Network*. Cham: Springer International Publishing, 2017, pp. 155–169. ISBN 978-3-319-60633-0.

17. CHECA, David; BUSTILLO, Andres. A Review of Immersive Virtual Reality Serious Games to Enhance Learning and Training. *Multimedia Tools and Applications*. 2020, vol. 79, no. 9, pp. 5501–5527. ISSN 1573-7721. Available from doi: 10.1007/s11042-019-08348-9.
18. FUTUCLASS. *Educational VR Games*. 2023. Available also from: https://futuclass.com/blog/educational_vr_games/. Accessed: 2024-11-12.
19. DWIVEDI, Y.K.; KSHETRI, N.; HUGHES, L.; SLADE, E.L.; JEYARAJ, A.; KAR, A.K.; BAABDULLAH, A.M.; KOOHANG, A.; RAGHAVAN, V.; AHUJA, M. "So what if ChatGPT wrote it?" Multidisciplinary Perspectives on Opportunities, Challenges and Implications of Generative Conversational AI for Research, Practice and Policy. *International Journal of Information Management*. 2023, vol. 71, p. 102642.
20. WANG, Zichong; CHU, Zhibo; DOAN, Thang Viet; NI, Shiwen; YANG, Min; ZHANG, Wenbin. History, Development, and Principles of Large Language Models: An Introductory Survey. *AI and Ethics*. 2024. ISSN 2730-5961. Available from doi: 10.1007/s43681-024-00583-7.
21. HADI, Muhammad Usman; TASHI, Qasem Al; QURESHI, Rizwan; AL., et. A Survey on Large Language Models: Applications, Challenges, Limitations, and Practical Usage. *Business and Information Systems Engineering*. 2023. Available from doi: 10.36227/techrxiv.23589741.v1.
22. TEUBNER, T.; FLATH, C.M.; WEINHARDT, C., et al. Welcome to the Era of ChatGPT et al. *Business and Information Systems Engineering*. 2023, vol. 65, no. 2, pp. 95–101. Available from doi: 10.1007/s12599-023-00795-x.
23. TEAM, OpenAI. *ChatGPT: Optimizing language models for dialogue*. 2022.
24. OPENAI. *ChatGPT*. 2024. Available also from: <https://openai.com/chatgpt/>. Accessed: 2024-12-10.
25. OPENAI. [N.d.]. Available also from: <https://openai.com/>.

26. WIKIPEDIA CONTRIBUTORS. *ChatGPT — Wikipedia, The Free Encyclopedia*. 2024. Available also from: <https://en.wikipedia.org/w/index.php?title=ChatGPT&oldid=1256553544>. [Online; accessed 11-November-2024].
27. LI, Jianning; DADA, Amin; PULADI, Behrus; KLEESIEK, Jens; EGGER, Jan. ChatGPT in healthcare: A taxonomy and systematic review. *Computer Methods and Programs in Biomedicine*. 2024, vol. 245, p. 108013. issn 0169-2607. Available from doi: <https://doi.org/10.1016/j.cmpb.2024.108013>.
28. GUINNESS, Harry. *The best large language models (llms) in 2024*. Zapier, 2024. Available also from: <https://zapier.com/blog/best-llm/>.
29. LUTKEVICH, Ben. *19 of the best large language models in 2024*. TechTarget, 2024. Available also from: <https://www.techtarget.com/what-is/feature/12-of-the-best-large-language-models>.
30. SALLAM, M. The utility of ChatGPT as an example of large language models in healthcare education, research, and practice: Systematic review on the future perspectives and potential limitations. *medRxiv*. 2023, pp. 2023–02.
31. WU, S.; IRSOY, O.; LU, S.; DABRAVOLSKI, V.; DREDZE, M.; GEHRMANN, S.; KAMBADUR, P.; ROSENBERG, D.; MANN, G. BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*. 2023.
32. CHEN, M.; TWOREK, J.; JUN, H.; YUAN, Q.; O. PINTO, H. P. d.; KAPLAN, J.; EDWARDS, H.; BURDA, Y.; JOSEPH, N.; BROCKMAN, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*. 2021.
33. WU, Xiaodong; DUAN, Ran; NI, Jianbing. Unveiling security, privacy, and ethical concerns of ChatGPT. *Journal of Information and Intelligence*. 2024, vol. 2, no. 2, pp. 102–115. issn 2949-7159. Available from doi: <https://doi.org/10.1016/j.jiixd.2023.10.007>.

34. ALKAMLI, Shahad; ALABDULJABBAR, Reham. Understanding privacy concerns in CHATGPT: A data-driven approach with LDA Topic Modeling. *Heliyon*. 2024, vol. 10, no. 20. Available from DOI: 10.1016/j.heliyon.2024.e39087.
35. YAN, Lixiang; SHA, Lele; ZHAO, Linxuan; LI, Yuheng; MARTINEZ-MALDONADO, Roberto; CHEN, Guanliang; LI, Xinyu; JIN, Yueqiao; GAŠEVIĆ, Dragan. Practical and ethical challenges of large language models in education: A systematic scoping review. *British Journal of Educational Technology*. 2023, vol. 55, no. 1, pp. 90–112. ISSN 1467-8535. Available from DOI: 10.1111/bjet.13370.
36. ZIRAR, Araz. Exploring the impact of language models, such as ChatGPT, on student learning and assessment. *Review of Education*. 2023, vol. 11, no. 3, e3433. Available from DOI: <https://doi.org/10.1002/rev3.3433>. e3433 RoE-2023-06-0860.R1.
37. SALMINEN, Joni; JUNG, Soon-gyo; MEDINA, Johanne; ALDOUS, Kholoud; AZEM, Jinan; AKHTAR, Waleed; JANSEN, Bernard J. Using Cipherbot: An Exploratory Analysis of Student Interaction with an LLM-Based Educational Chatbot. In: *Proceedings of the Eleventh ACM Conference on Learning @ Scale*. Atlanta, GA, USA: Association for Computing Machinery, 2024, pp. 279–283. L@S '24. ISBN 9798400706332. Available from DOI: 10.1145/3657604.3664690.
38. DAN, Yuhao; LEI, Zhikai; GU, Yiyang; LI, Yong; YIN, Jianghao; LIN, Jiaju; YE, Linhao; TIE, Zhiyan; ZHOU, Yougen; WANG, Yilei; ZHOU, Aimin; ZHOU, Ze; CHEN, Qin; ZHOU, Jie; HE, Liang; QIU, Xipeng. *EduChat: A Large-Scale Language Model-based Chatbot System for Intelligent Education*. 2023. Available from arXiv: 2308.02773 [cs.CL].
39. ZHANG, Zheyuan; ZHANG-LI, Daniel; YU, Jifan; GONG, Linlu; ZHOU, Jinchang; LIU, Zhiyuan; HOU, Lei; LI, Juanzi. *Simulating Classroom Education with LLM-Empowered Agents*. 2024. Available from arXiv: 2406.19226 [cs.CL].
40. DIJKSTRA, R.; GENÇ, Z.; KAYAL, S.; KAMPS, J. Reading Comprehension Quiz Generation using Generative Pre-trained Transformers. In: SOSNOVSKY, S.; BRUSILOVSKY, P.; LAN, A. (eds.).

- Proceedings of the Fourth International Workshop on Intelligent Text-books 2022, co-located with the 23rd International Conference on Artificial Intelligence in Education (AIED 2022)*. Durham, UK: CEUR-WS, 2022, vol. 3192, pp. 4–17. CEUR Workshop Proceedings. Available also from: <https://hdl.handle.net/11245.1/a1109043-92d4-4c63-be33-6e238780d3b7>.
41. BHAT, Shravya; NGUYEN, Huy Anh; MOORE, Steven; STAMPER, John C; SAKR, Majd; NYBERG, Eric. Towards Automated Generation and Evaluation of Questions in Educational Domains. In: *EDM*. 2022.
 42. KUNG, Tiffany H.; CHEATHAM, Morgan; MEDENILLA, Arielle; SILLOS, Czarina; DE LEON, Lorie; ELEPAÑO, Camille; MADRIAGA, Maria; AGGABAO, Rimel; DIAZ-CANDIDO, Giezel; MANINGO, James; TSENG, Victor. Performance of ChatGPT on USMLE: Potential for AI-assisted medical education using large language models. *PLOS Digital Health*. 2023, vol. 2, no. 2, pp. 1–12. Available from doi: 10.1371/journal.pdig.0000198.
 43. OH, Namkee; CHOI, Gyu-Seong; LEE, Woo Yong. ChatGPT Goes to Operating Room: Evaluating GPT-4 Performance and Its Potential in Surgical Education and Training in the Era of Large Language Models. *medRxiv*. 2023. Available from doi: 10.1101/2023.03.16.23287340.
 44. BAO, Minhui. Can Home Use of Speech-Enabled Artificial Intelligence Mitigate Foreign Language Anxiety – Investigation of a Concept. *Arab World English Journal (AWEJ) Special Issue on CALL*. 2019, vol. 5. Available from doi: 10.2139/ssrn.3431734. Available at SSRN: <https://ssrn.com/abstract=3431734>.
 45. JIA, Qinjin; CUI, Jialin; XIAO, Yunkai; LIU, Chengyuan; RASHID, Parvez; GEHRINGER, Edward F. *ALL-IN-ONE: Multi-Task Learning BERT models for Evaluating Peer Assessments*. 2021. Available from arXiv: 2110.03895 [cs.CL].
 46. JAEKWON PARK; JIYOUNG BAE; UNGGI LEE; TAEKYUNG AHN; SOOKBUN LEE; DOHEE KIM; CHOI, Aram; YEIL JEONG; JEWON MOON; HYEONCHEOL KIM. How to Align Large Language Models for Teaching English? Designing and Developing LLM based-Chatbot for Teaching English Conversa-

- tion in EFL, Findings and Limitations. 2024. Available from doi: 10.13140/RG.2.2.13490.82883.
47. ALLARD, Marc-Antoine; ANSARIPOUR, Matin; YUFFA, Maria; TEILETCHE, Paul. *LLaMa-SciQ: An Educational Chatbot for Answering Science MCQ*. 2024. Available from arXiv: 2409.16779 [cs.AI].
 48. SWEETSER, Penny. Large Language Models and Video Games: A Preliminary Scoping Review. In: *Proceedings of the 6th ACM Conference on Conversational User Interfaces*. Luxembourg, Luxembourg: Association for Computing Machinery, 2024. CUI '24. ISBN 9798400705113. Available from doi: 10.1145/3640794.3665582.
 49. ZHU, Xizhou; CHEN, Yuntao; TIAN, Hao; TAO, Chenxin; SU, Weijie; YANG, Chenyu; HUANG, Gao; LI, Bin; LU, Lewei; WANG, Xiaogang; QIAO, Yu; ZHANG, Zhaoxiang; DAI, Jifeng. *Ghost in the Minecraft: Generally Capable Agents for Open-World Environments via Large Language Models with Text-based Knowledge and Memory*. 2023. Available from arXiv: 2305.17144 [cs.AI].
 50. WANG, Guanzhi; XIE, Yuqi; JIANG, Yunfan; MANDLEKAR, Ajay; XIAO, Chaowei; ZHU, Yuke; FAN, Linxi; ANANDKUMAR, Anima. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023. Available from arXiv: 2305.16291 [cs.AI].
 51. BRAWER, Jake; BISHOP, Kayleigh; HAYES, Bradley; RONCONE, Alessandro. *Towards A Natural Language Interface for Flexible Multi-Agent Task Assignment*. 2023. Available from arXiv: 2311.00153 [cs.R0].
 52. VERMA, Mudit; BHAMBRI, Siddhant; KAMBHAMPATI, Subbarao. Preference Proxies: Evaluating Large Language Models in capturing Human Preferences in Human-AI Tasks. In: *First Workshop on Theory of Mind in Communicating Agents*. 2023. Available also from: <https://openreview.net/forum?id=m6EpkjUUBR>.
 53. SIDJI, Matthew; SMITH, Wally; ROGERSON, Melissa J. Human-AI Collaboration in Cooperative Games: A Study of Playing Codenames with an LLM Assistant. *Proc. ACM Hum.-Comput. Interact.* 2024, vol. 8, no. CHI PLAY. Available from doi: 10.1145/3677081.

54. AFONSO, Nuno; PRADA, Rui. Agents That Relate: Improving the Social Believability of Non-Player Characters in Role-Playing Games. In: STEVENS, Scott M.; SALDAMARCO, Shirley J. (eds.). *Entertainment Computing - ICEC 2008*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 34–45. ISBN 978-3-540-89222-9.
55. ROGERS, Katja; AUFHEIMER, Maria; WEBER, Michael; NACKE, Lennart E. Exploring the Role of Non-Player Characters and Gender in Player Identification. In: *Proceedings of the 2018 Annual Symposium on Computer-Human Interaction in Play Companion Extended Abstracts*. Melbourne, VIC, Australia: Association for Computing Machinery, 2018, pp. 271–283. CHI PLAY '18 Extended Abstracts. ISBN 9781450359689. Available from doi: 10.1145/3270316.3273041.
56. JOHANSSON, Magnus; STRÅÅT, Björn; WARPEFELT, Henrik; VERHAGEN, Harko. Analyzing the Social Dynamics of Non-Player Characters. In: MEIJER, Sebastiaan A.; SMEDS, Riitta (eds.). *Frontiers in Gaming Simulation*. Cham: Springer International Publishing, 2014, pp. 173–187. ISBN 978-3-319-04954-0.
57. LANKOSKI, Petri; BJÖRK, Staffan. Gameplay Design Patterns for Believable Non-Player Characters. In: 2007.
58. MARINCIONI, Alessandro; MILTIADOUS, Myriana; ZACHARIA, Katerina; HEEMSKERK, Rick; DOUKERIS, Georgios; PREUSS, Mike; BARBERO, Giulio. The effect of LLM-based NPC emotional states on player emotions: An analysis of interactive game play. *2024 IEEE Conference on Games (CoG)*. 2024, pp. 1–6. Available from doi: 10.1109/cog60054.2024.10645631.
59. PENG, Xiangyu; QUAYE, Jessica; RAO, Sudha; XU, Weijia; BOTCHWAY, Portia; BROCKETT, Chris; JOJIC, Nebojsa; DESGARENNES, Gabriel; LOBB, Ken; XU, Michael; LEANDRO, Jorge; JIN, Claire; DOLAN, Bill. Player-Driven Emergence in LLM-Driven Game Narrative. In: *2024 IEEE Conference on Games (CoG)*. 2024, pp. 1–8. Available from doi: 10.1109/CoG60054.2024.10645607.
60. YONG, Qing Ru; MITCHELL, Alex. From Playing the Story to Gaming the System: Repeat Experiences of a Large Language Model-Based Interactive Story. In: HOLLOWAY-ATTAWAY, Lissa;

- MURRAY, John T. (eds.). *Interactive Storytelling*. Cham: Springer Nature Switzerland, 2023, pp. 395–409. ISBN 978-3-031-47655-6.
61. LI, Cheng; LENG, Ziang; YAN, Chenxi; SHEN, Junyi; WANG, Hao; MI, Weishi; FEI, Yaying; FENG, Xiaoyang; YAN, Song; WANG, HaoSheng; ZHAN, Linkang; JIA, Yaokai; WU, Pingyu; SUN, Haozhen. *ChatHaruhi: Reviving Anime Character in Reality via Large Language Model*. 2023. Available from arXiv: 2308.09597 [cs.CL].
 62. COX, Samuel Rhys; OOI, Wei Tsang. Conversational Interactions with NPCs in LLM-Driven Gaming: Guidelines from a Content Analysis of Player Feedback. In: *Chatbot Research and Design: 7th International Workshop, CONVERSATIONS 2023, Oslo, Norway, November 22–23, 2023, Revised Selected Papers*. Oslo, Norway: Springer-Verlag, 2024, pp. 167–184. ISBN 978-3-031-54974-8. Available from doi: 10.1007/978-3-031-54975-5_10.
 63. SHAO, Yunfan; LI, Linyang; DAI, Junqi; QIU, Xipeng. *Character-LLM: A Trainable Agent for Role-Playing*. 2023. Available from arXiv: 2310.10158 [cs.CL].
 64. ALI, Safinah; PARK, Hae Won; BREAZEAL, Cynthia. Can Children Emulate a Robotic Non-Player Character’s Figural Creativity? In: *Proceedings of the Annual Symposium on Computer-Human Interaction in Play*. Virtual Event, Canada: Association for Computing Machinery, 2020, pp. 499–509. CHI PLAY ’20. ISBN 9781450380744. Available from doi: 10.1145/3410404.3414251.
 65. RAO, Sudha; XU, Weijia; XU, Michael; LEANDRO, Jorge; LOBB, Ken; DESGARENNES, Gabriel; BROCKETT, Chris; DOLAN, Bill. *Collaborative Quest Completion with LLM-driven Non-Player Characters in Minecraft*. 2024. Available from arXiv: 2407.03460 [cs.CL].
 66. PARK, Joon Sung; O’BRIEN, Joseph C.; CAI, Carrie J.; MORRIS, Meredith Ringel; LIANG, Percy; BERNSTEIN, Michael S. *Generative Agents: Interactive Simulacra of Human Behavior*. 2023. Available from arXiv: 2304.03442 [cs.HC].

-
67. AKSHIT IREDDY. *Interactive-LLM-Powered-NPCs*. 2023. Available also from: <https://github.com/AkshitIreddy/Interactive-LLM-Powered-NPCs?tab=readme-ov-file>. Accessed: 2024-12-10.
 68. IREDDY, Akshit. *Talk to GPT-4 powered npcs in your favorite games!* Medium, 2024. Available also from: <https://medium.com/@akshit.r.ireddy/talk-to-gpt-4-powered-npcs-in-your-favorite-games-e508d1661b80>.
 69. SEAL-FOSS, Reed. *Integrating large language models like Open Ai's GPT with unity 3D*. Medium, 2023. Available also from: <https://medium.com/@ReedSealFoss/integrating-large-language-models-like-open-ais-gpt-with-unity-3d-c4e2faf2e82b>.
 70. PROXIMA ENTERPRISES INC. *SuckUp!* 2024. Available also from: <https://www.playsuckup.com/>. Accessed: 2024-12-10.
 71. GOODAI GAMES S.R.O. *AI People Game*. 2024. Available also from: <https://www.aipeoplegame.com/>. Accessed: 2024-12-10.
 72. LEGEND HAS IT. *Realms of Alterra*. 2024. Available also from: https://store.steampowered.com/app/2938420/Realms_of_Alterra/. Accessed: 2024-12-10.
 73. 2024. Available also from: <https://www.goodai.com/ai-people-announcing-the-next-evolution-of-gaming-ai-npcs/>.
 74. MANČAŘ, Michal. *Češi tvoří hru, ve které se s Každou postavou bavíte Jako S Umělou inteligencí chatgpt*. 2023. Available also from: <https://cc.cz/cesi-vyvi-ji-hru-ve-kt-re-se-s-kazdou-postavou-bavite-jako-s-umelou-inteligenci-chatgpt/>.
 75. CONVAI TECHNOLOGIES INC. *Convai: Conversational AI Platform*. 2024. Available also from: <https://convai.com>. Accessed: 2024-12-10.
 76. CHEN, Liuqing; CAI, Yu; WANG, Ruyue; DING, Shixian; TANG, Yilin; HANSEN, Preben; SUN, Lingyun. Supporting Text Entry in Virtual Reality with Large Language Models. In: *2024 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*. 2024, pp. 524–534. Available from doi: 10.1109/VR58804.2024.00073.

-
77. BULDU, Kadir Burak; ÖZDEL, Süleyman; LAU, Ka Hei Carrie; WANG, Mengdi; SAAD, Daniel; SCHÖNBORN, Sofie; BOCH, Auxane; KASNECI, Enkelejda; BOZKIR, Efe. *CUIfy the XR: An Open-Source Package to Embed LLM-powered Conversational Agents in XR*. 2024. Available from arXiv: 2411.04671 [cs.HC].
 78. HAYDEN, Scott. *This "Skyrim VR" mod uses CHATGPT to bring Tamriel's npcs to life*. 2023. Available also from: <https://www.roadtovr.com/chatgpt-skyrim-vr-ai-mod/>.
 79. KYRLITSIAS, Christos; MICHAEL-GRIGORIOU, Despina. Social Interaction With Agents and Avatars in Immersive Virtual Environments: A Survey. *Frontiers in Virtual Reality*. 2022, vol. 2. ISSN 2673-4192. Available from DOI: 10.3389/frvir.2021.786665.
 80. SUZUKI, Ryo; GONZALEZ-FRANCO, Mar; SRA, Misha; LINDLBAUER, David. XR and AI: AI-Enabled Virtual, Augmented, and Mixed Reality. In: *Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. San Francisco, CA, USA: Association for Computing Machinery, 2023. UIST '23 Adjunct. ISBN 9798400700965. Available from DOI: 10.1145/3586182.3617432.
 81. YOUSRI, Ramez; ESSAM, Zeyad; KAREEM, Yehia; SHERIEF, Youstina; GAMIL, Sherry; SAFWAT, Soha. *IllusionX: An LLM-powered mixed reality personal companion*. 2024. Available from DOI: 10.36227/techrxiv.170775603.33253596/v1.
 82. BOZKIR, Efe; ÖZDEL, Süleyman; LAU, Ka Hei Carrie; WANG, Mengdi; GAO, Hong; KASNECI, Enkelejda. Embedding Large Language Models into Extended Reality: Opportunities and Challenges for Inclusion, Engagement, and Privacy. In: *ACM Conversational User Interfaces 2024*. ACM, 2024. CUI'24. Available from DOI: 10.1145/3640794.3665563.
 83. WAN, Hongyu; ZHANG, Jinda; SURIA, Abdulaziz Arif; YAO, Bingsheng; WANG, Dakuo; COADY, Yvonne; PRPA, Mirjana. Building LLM-based AI Agents in Social Virtual Reality. In: *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. Honolulu, HI, USA: Association for Computing

- Machinery, 2024. CHI EA '24. ISBN 9798400703317. Available from doi: 10.1145/3613905.3651026.
84. TEDDYBEAR082. *godot4-ai-npc-example*. 2023. Available also from: github.com/teddybear082/godot4-ai-npc-example. Accessed: 2024-12-10.
 85. ART FROM THE MACHINE. *Mantella - Bring NPCs to Life with AI*. 2023. Available also from: <https://www.nexusmods.com/skyrimspecialiedition/mods/98631?tab=posts>. Accessed: 2024-12-10.
 86. YUNANTO, Andhik Ampuh; HERUMURTI, Darlis; ROCHIMAH, Siti; KUSWARDAYAN, Imam. English Education Game using Non-Player Character Based on Natural Language Processing. *Procedia Computer Science*. 2019, vol. 161, pp. 502–508. ISSN 1877-0509. Available from doi: <https://doi.org/10.1016/j.procs.2019.11.158>. The Fifth Information Systems International Conference, 23-24 July 2019, Surabaya, Indonesia.
 87. GOSLEN, Alex; KIM, Yeo Jin; ROWE, Jonathan; LESTER, James. LLM-Based Student Plan Generation for Adaptive Scaffolding in Game-Based Learning Environments. *International Journal of Artificial Intelligence in Education*. 2024. Available from doi: 10.1007/s40593-024-00421-1.
 88. LAU, Ka Hei Carrie; BOZKIR, Efe; GAO, Hong; KASNECI, Enkelejda. *Evaluating Usability and Engagement of Large Language Models in Virtual Reality for Traditional Scottish Curling*. 2024. Available from arXiv: 2408.09285 [cs.HC].
 89. IZQUIERDO-DOMENECH, Juan; LINARES-PELLICER, Jordi; FERRI-MOLLA, Isabel. Virtual reality and language models, a new frontier in learning. *International Journal of Interactive Multimedia and Artificial Intelligence*. 2024, vol. 8, no. 5, p. 46. Available from doi: 10.9781/ijimai.2024.02.007.
 90. WIKIPEDIA CONTRIBUTORS. *Llama (language model)* — Wikipedia, *The Free Encyclopedia*. 2024. Available also from: [https://en.wikipedia.org/w/index.php?title=Llama_\(language_model\)&oldid=1257442456](https://en.wikipedia.org/w/index.php?title=Llama_(language_model)&oldid=1257442456). [Online; accessed 16-November-2024].

91. BAIER, Fabian. 2023: *A year in Review – pulze, the evolution and impact of Generative AI, and beyond*. Pulze.ai Inc., 2023. Available also from: <https://blog.pulze.ai/2023-in-generative-ai-comprehensive-recap/>.
92. HUBER, Stefan E.; KIILLI, Kristian; NEBEL, Steve; RYAN, Richard M.; SAILER, Michael; NINAUS, Manuel. Leveraging the Potential of Large Language Models in Education Through Playful and Game-Based Learning. *Educational Psychology Review*. 2024, vol. 36, no. 1, p. 25. Available from doi: 10.1007/s10648-024-09868-z.
93. GIBBS, Joe. *Local LLM-powered npcs with llama.cpp, MISTRAL7B and styletts2*. [N.d.]. Available also from: <https://jgibbs.dev/blogs/local-llm-npcs-in-unreal-engine>.
94. PACKER, Charles; WOODERS, Sarah; LIN, Kevin; FANG, Vivian; PATIL, Shishir G.; STOICA, Ion; GONZALEZ, Joseph E. *MemGPT: Towards LLMs as Operating Systems*. 2024. Available from arXiv: 2310.08560 [cs.AI].
95. OPENAI. [N.d.]. Available also from: <https://openai.com/index/memory-and-new-controls-for-chatgpt>.
96. [N.d.]. Available also from: <https://www.snowflake.com/guides/llm-inference/#:~:text=LLM%20inference%20is%20the%20mechanism,those%20tokens%20into%20text%20outputs..>
97. WU, Sean; KOO, Michael; BLUM, Lesley; BLACK, Andy; KAO, Liyo; SCALZO, Fabien; KURTZ, Ira. *A Comparative Study of Open-Source Large Language Models, GPT-4 and Claude 2: Multiple-Choice Test Taking in Nephrology*. 2023. Available from arXiv: 2308.04709 [cs.CL].
98. TUGGENER, Lukas; SAGER, Pascal; TAOUDI-BENCHEKROUN, Yassine; GREWE, Benjamin F.; STADELMANN, Thilo. So you want your private LLM at home? A survey and benchmark of methods for efficient gpts. *2024 11th IEEE Swiss Conference on Data Science (SDS)*. 2024, pp. 205–212. Available from doi: 10.1109/sds60720.2024.00036.

99. WOLF, Thomas; DEBUT, Lysandre; SANH, Victor; CHAUMOND, Julien; DELANGUE, Clement; MOI, Anthony; CISTAC, Pierrick; RAULT, Tim; LOUF, Rémi; FUNTOWICZ, Morgan; DAVISON, Joe; SHLEIFER, Sam; PLATEN, Patrick von; MA, Clara; JERNITE, Yacine; PLU, Julien; XU, Canwen; SCAO, Teven Le; GUGGER, Sylvain; DRAME, Mariama; LHOEST, Quentin; RUSH, Alexander M. *HuggingFace's Transformers: State-of-the-art Natural Language Processing*. 2020. Available from arXiv: 1910.03771 [cs.CL].
100. HUGGING FACE. *Hugging Face*. 2016. Available also from: <https://huggingface.co/>. Accessed: 2024-12-10.
101. DANIEL HAN, Michael Han. *Unsloth*. 2023. Available also from: <https://unsloth.ai/>. Accessed: 2024-12-10.
102. PEDREGOSA, F.; VAROQUAUX, G.; GRAMFORT, A.; MICHEL, V.; THIRION, B.; GRISEL, O.; BLONDEL, M.; PRETTENHOFER, P.; WEISS, R.; DUBOURG, V.; VANDERPLAS, J.; PASSOS, A.; COURNAPEAU, D.; BRUCHER, M.; PERROT, M.; DUCHESNAY, E. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*. 2011, vol. 12, pp. 2825–2830.
103. RUNPOD. *RunPod Documentation*. 2022. Available also from: <https://docs.runpod.io/overview>. Accessed: 2024-12-10.
104. PASZKE, Adam; GROSS, Sam; MASSA, Francisco; LERER, Adam; BRADBURY, James; CHANAN, Gregory; KILLEEN, Trevor; LIN, Zeming; GIMELSHEIN, Natalia; ANTIGA, Luca; DESMAISON, Alban; KÖPF, Andreas; YANG, Edward; DEVITO, Zach; RAISON, Martin; TEJANI, Alykhan; CHILAMKURTHY, Sasank; STEINER, Benoit; FANG, Lu; BAI, Junjie; CHINTALA, Soumith. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. Available from arXiv: 1912.01703 [cs.LG].
105. GITHUB. *GitHub Docs*. 2008. Available also from: <https://docs.github.com/en>. Accessed: 2024-12-10.
106. JUPYTER, Project. *Jupyter*. 2008. Available also from: <https://jupyter.org/>. Accessed: 2024-12-10.

107. MANDAL, Sucheta. *Understanding base models and instruct models in hugging face*. Medium, 2024. Available also from: <https://medium.com/@sucheta963/understanding-base-models-and-instruct-models-in-hugging-face-4de0ad0d858c>.
108. CHAMBERS, Mike. *What is an instruct model? - instruction and chat fine-tuning*. [N.d.]. Available also from: <https://community.aws/content/2ZVa61RxToXUFzcuY8Hbut6L150/what-is-an-instruct-model?lang=en>.
109. WALKER II, Stephen M. *Best open source llms of 2024*. 2024. Available also from: <https://klu.ai/blog/open-source-llm-models>.
110. STOELINGA, Sam. *Calculating GPU memory for serving LLMS*. 2023. Available also from: <https://www.substratus.ai/blog/calculating-gpu-memory-for-llm>.
111. TEAM, Mistral AI. *Mistral-7B-Instruct-v0.3*. 2024. Available also from: <https://deepinfra.com/mistralai/Mistral-7B-Instruct-v0.3/api>.
112. TEAM, Gemma. *Gemma 2: Improving Open Language Models at a Practical Size*. 2024. Available from arXiv: 2408.00118 [cs.CL].
113. GRATTAFIORI, Aaron et al. *The Llama 3 Herd of Models*. 2024. Available from arXiv: 2407.21783 [cs.AI].
114. TEAM, Mistral AI. *Mistral Nemo Instruct*. 2024. Available also from: <https://huggingface.co/mistralai/Mistral-Nemo-Instruct-2407>.
115. ZHANG, Peiyuan; ZENG, Guangtao; WANG, Tianduo; LU, Wei. *TinyLlama: An Open-Source Small Language Model*. 2024. Available from arXiv: 2401.02385 [cs.CL].
116. META. *Llama 3.2: Revolutionizing edge AI and vision with open, customizable models*. 2024. Available also from: <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>.
117. CHOUDHURY, Sohan. *What is a system prompt?* 2024. Available also from: <https://help.flintk12.com/en/articles/9025167-what-is-a-system-prompt>.

118. CORPORATION, IBM. [N.d.]. Available also from: <https://www.ibm.com/docs/en/watsonx/saas?topic=solutions-tokens>.
119. HULSE, Spencer. *More words, less accuracy: The surprising impact of prompt length on LLM Performance*. 2024. Available also from: <https://gritdaily.com/impact-prompt-length-llm-performance/>.
120. WIKIPEDIA CONTRIBUTORS. *Fine-tuning (deep learning)* — *Wikipedia, The Free Encyclopedia*. 2024. Available also from: [https://en.wikipedia.org/w/index.php?title=Fine-tuning_\(deep_learning\)&oldid=1255690425](https://en.wikipedia.org/w/index.php?title=Fine-tuning_(deep_learning)&oldid=1255690425). [Online; accessed 26-November-2024].
121. PARTHASARATHY, Venkatesh Balavadhani; ZAFAR, Ahtsham; KHAN, Aafaq; SHAHID, Arsalan. *The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs: An Exhaustive Review of Technologies, Research, Best Practices, Applied Research Challenges and Opportunities*. 2024. Available from arXiv: 2408.13296 [cs.LG].
122. HEIDLOFF, Niklas. *Instruction fine-tuning of large language models*. 2023. Available also from: <https://heidloff.net/article/instruct-tuning-large-language-models/>.
123. LABONNE, Maxime. *Fine-tune llama 3.1 ultra-efficiently with Unsloth*. 2024. Available also from: <https://huggingface.co/blog/mlabonne/sft-llama3>.
124. HU, Edward J.; SHEN, Yelong; WALLIS, Phillip; ALLEN-ZHU, Zeyuan; LI, Yuanzhi; WANG, Shean; WANG, Lu; CHEN, Weizhu. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. Available from arXiv: 2106.09685 [cs.CL].
125. DETTMERS, Tim; PAGNONI, Artidoro; HOLTZMAN, Ari; ZETTLEMOYER, Luke. *QLoRA: Efficient Finetuning of Quantized LLMs*. 2023. Available from arXiv: 2305.14314 [cs.LG].
126. BERGMANN, Dave. *What is instruction tuning?* 2024. Available also from: <https://www.ibm.com/topics/instruction-tuning>.

127. VAKAYIL, Sonia; JULIET, D. Sujitha; J, Anitha.; VAKAYIL, Sunil. RAG-Based LLM Chatbot Using Llama-2. In: *2024 7th International Conference on Devices, Circuits and Systems (ICDCS)*. 2024, pp. 1–5. Available from doi: 10.1109/ICDCS59278.2024.10561020.
128. SILVA, Daswin; MILLS, Nishan; EL-AYOUBI, Mona; MANIC, Milos; ALAHAKOON, Damminda. ChatGPT and Generative AI Guidelines for Addressing Academic Integrity and Augmenting Pre-Existing Chatbots. In: 2023, pp. 1–6. Available from doi: 10.1109/ICIT58465.2023.10143123.
129. GAMAGE, Gihan; MILLS, Nishan; DE SILVA, Daswin; MANIC, Milos; MORALIYAGE, Harsha; JENNINGS, Andrew; ALAHAKOON, Damminda. Multi-Agent RAG Chatbot Architecture for Decision Support in Net-Zero Emission Energy Systems. In: *2024 IEEE International Conference on Industrial Technology (ICIT)*. 2024, pp. 1–6. Available from doi: 10.1109/ICIT58233.2024.10540920.
130. [N.d.]. Available also from: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>.
131. RENDEVSKI, Nikola; TRAJCEVSKA, Diana; DIMOVSKI, Mario; VELJANOVSKI, Konstantin; POPOV, Aleksandar; EMINI, Naile; VELJANOVSKI, Dimitar. PC VR vs Standalone VR Fully-Immersive Applications: History, Technical Aspects and Performance. In: *2022 57th International Scientific Conference on Information, Communication and Energy Systems and Technologies (ICEST)*. 2022, pp. 1–4. Available from doi: 10.1109/ICEST55168.2022.9828656.
132. WIKIPEDIA CONTRIBUTORS. *Quest 2 — Wikipedia, The Free Encyclopedia*. 2024. Available also from: https://en.wikipedia.org/w/index.php?title=Quest_2&oldid=1259560418. [Online; accessed 3-December-2024].
133. GRINBERG, Miguel. *Flask web development: developing web applications with python*. " O'Reilly Media, Inc.", 2018.
134. MERKEL, Dirk. Docker: lightweight linux containers for consistent development and deployment. *Linux journal*. 2014, vol. 2014, no. 239, p. 2.

135. UNITY TECHNOLOGIES. *Unity*. 2022. 2022.3.50f. Available also from: <https://unity.com/>. Game development platform.
136. UNITY. *XR Interaction Toolkit: XR interaction toolkit: 3.0.7*. [N.d.]. Available also from: <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.0/manual/index.html>.
137. UNITY. *VR Game Development Software & Engine*. [N.d.]. Available also from: <https://unity.com/solutions/vr>.
138. META HORIZON. *Voice SDK*. 2024. Available also from: <https://developers.meta.com/horizon/documentation/unity/voice-sdk-overview/>. Accessed: 2024-12-10.
139. WIT.AI. *Getting Started With Wit.ai*. [N.d.]. Available also from: <https://wit.ai/docs>.
140. AMPLIFY CREATIONS. *FXAA Fast Approximate Anti-Aliasing*. 2012. Version 1.5. Available also from: <https://assetstore.unity.com/packages/vfx/shaders/fullscreen-camera-effects/fxaa-fast-approximate-anti-aliasing-3590#version-current>. Accessed: 2024-11-10.
141. NIKOLAI ZHOLUDEV. *Robot from the series "Love death and robots"*. 2021. Available also from: <https://sketchfab.com/3d-models/robot-from-the-series-love-death-and-robots-3c2c43c8737341e5816c61634782373a>. Accessed: 2024-12-03.
142. DOWNTOWN GAME STUDIO. *HYPERCASUAL - Simple Female & Male Characters*. 2022. Available also from: <https://assetstore.unity.com/packages/3d/characters/hypercasual-simple-female-male-characters-209163#releases>. Accessed: 2024-12-14.
143. LEE, Huei; ZHANG, Yu; CHEN, Kuo Lane. The Use of Virtual Reality for Real Estate and Hospitality Management. *International Journal of Management Theory and Practices*. 2024, vol. 19, no. 1, pp. 46–58.
144. TECHNOLOGIES, Unity. *Adventure Game Environment Pack: URP: 3D Sci-Fi*. [N.d.]. Available also from: <https://assetstore.unity.com/packages/3d/environments/sci-fi/adventure-game-environment-pack-urp-246189>.

BIBLIOGRAPHY

145. BLACK WHALE STUDIO. *VR Keyboard - XRI Poke & Hands*. 2023. Version 1.2. Available also from: <https://blackwhale.dev/portfolio/free-vr-keyboard-asset/>. Accessed: 2024-11-08.
146. BRŮŽA, Vojtěch; ROSECKÝ, Jonáš. *Emcify*. [N.d.]. Available also from: <https://brno.emcify.cz/en/about>. Accessed: 2024-12-13.
147. CHRIS NOLET. *Quick Outline*. 2018. Version 1.1. Available also from: <https://assetstore.unity.com/packages/tools/particles-effects/quick-outline-115488>. Accessed: 2024-11-09.

A Electronic Attachments

The following attachments are stored in the archive of IS MU:

1. *LLM-NPC-Server.zip* contains the containerized application for the LLM Server component. A *ReadMe.txt* document is included, outlining all necessary steps and requirements for running the server locally.
2. *QuestForInformation-Unity.zip* contains the source code for the Unity Game Client. It also contains a video (*DemoVideo.MP4*) showcasing the gameplay as well as the build of the game for Oculus Quest 2 (*build.apk*), which can be sideloaded onto the headset third-party applications.
3. *LLM-Customization.zip* tools used for the initial model comparison. It contains the question files used for testing, the complete test results, and Jupyter notebooks that contain helpful comments that can assist in recreating the testing process.
4. *LLM-Comparison.zip* contains tools for exploring different approaches to LLM customization. The code contains detailed comments, serving as a guide in recreating the same steps for any future use. It is provided in a containerized format, which can be run using Docker. A *ReadMe.txt* file is also included with further instructions for use.